3. ATALA

AA Frameworka

EGILEA

Aitor Aretxaga Agirre

ZENTROA

Uni Eibar-Ermua

EDIZIOA

v1.1 (Laneki — FP Euskadi materialean oinarritua)

IKASTAROA

2026 — 2027

AA Frameworka

Sarrera

Imajina ezazu astelehen goiz bat enpresa batean. Zure tutoreak mahaira hurbildu eta esan dizu: "Marketin-taldeak hilabeteetan zehar arazo bat izan du: Bezeroen iruzkin pila bat jaso ditugu eta inork ez du denak irakurtzeko denborarik. Bezeroak pozik daude ala haserre? Zein produktuk sortzen dute kexa gehien? Eta gainera, salmenta-taldeak galdetu du ea aurreikus dezakegun zein bezerok utziko gaituen hurrengo hilabetean". Ordu erdi geroago, beste eskaera bat dator: "Ah, eta zuzendaritzak nahi du dena web-orri batean ikustea klik batekin, eta API bat eduki dezagun beste sistemei datu horiek emateko".

Eskaera horiek guztiak —sentimenduen analisia, txandaketa-iragarpena, web-aplikazio interaktibo bat, API bat— gai honetan ikasiko dugun tresna-kutxarekin konpontzen dira. Ez dira ariketa akademikoak: enpresetan benetan eskatzen diren atazak dira, eta gaur egungo IA-teknikari batek egunero topatzen ditu.

Aurreko bi gaietan oinarriak jarri ditugu: Python ingurune profesionala (1. gaia) eta datu-zientziaren stack-a (2. gaia). Orain teknika horiek **produktu funtzional** bilakatuko ditugu. Ibilbide oso bat egingo dugu: **Scikit-Learn**ekin ML eredu klasikoak entrenatu, **Hugging Face**rekin eredu aurreentrenatu indartsuak erabili, **FastAPI**rekin API profesionalak eraiki, **Streamlit**ekin web-aplikazioak sortu, **LangChain + Gemini**rekin LLM aplikazioak garatu, eta azkenik **RAG** patroiarekin dokumentu propioei galderak egiteko sistema osatu. Amaieran, dena batuko dugu proiektu integratzaile batean.

> ⚠ ***Kontuz: ez dugu matematikara joko*** *Gai honetan ez ditugu algoritmoen formulak deribatuko. Helburua **tresnak ERABILTZEN** ikastea da, ez algoritmoak zerotik inplementatzea. Random Forest-ek nola funtzionatzen duen intuizioz ulertuko dugu (botoa ematen duten zuhaitz askoren batzarra), baina ez dugu ekuazio bakar bat ere idatziko. Industriak hori eskatzen dizu: tresna egokia une egokian aplikatzen jakitea.*

---

1. Scikit-Learn: Ikasketa Automatikoaren Programazioa

Atal honetan ikasketa automatiko klasikoa lantzeko Python-eko erreferentziazko liburutegia aztertuko dugu: **Scikit-Learn**. Helburua da ML-proiektu baten ohiko urratsak —datuak kargatu, aurreprozesatu, eredu bat aukeratu, entrenatu, ebaluatu eta hiperparametroak doitu— modu profesional eta erreproduzigarrian gauzatzen jakitea. Scikit-Learn menderatzea funtsezkoa da: enpresa-mailan zenbakizko datu egituratuekin lan egiteko (gaur egun, ekoizpen-sistemen ehuneko handi batek horrela egiten baitu lan oraindik), eta gerora ikusiko diren tresna aurreratuagoekin bateragarritasun zuzena duelako.

1.1 Zer da Scikit-Learn

> ☐ ***Errepasoa: 5072 moduluan ikusi duzue*** *Ikasketa automatikoaren ereduak (erregresio lineala/logistikoa, KNN, zuhaitzak, SVM, Random Forest, Ridge/Lasso)* ***5072 moduluan*** *landu dira sakon (`5072_2_Ikasketa_Gainbegiratua`), eta balioztatze-gurutzatua + GridSearchCV + data leakage `5072_3_Balidazio_Metodologia`-n. Hemen scikit-learn API-aren* ***erabilera profesionalean*** *zentratzen gara: `Pipeline`, `ColumnTransformer`, `joblib` eta ekoizpenerako zerbitzu-egiturak. Eredu bakoitzaren atzeko teoria ondo dakizula suposatzen dugu — falta bazaizu, kontsulta itzazu apunte horiek aurretik.*

Scikit-Learn (sklearn) Python-en ikasketa automatikorako liburutegi nagusia da. 2007an sortu zen, eta gaur egun industrian estandartzat hartzen da. Bere indar handiena interfaze bateratuan dago: eredu *guztiek* `fit()`, `predict()` eta `transform()` metodo berberak partekatzen dituzte. Horrek esan nahi du eredu bat beste batengatik aldatzeak ia ez duela koderik ukitzen — lerro bat soilik. Hori da abstrakzioaren botere erreala: behin patroia ikasita, ehunka eredu erabiltzen dakizu.

Instalazioa pakete-kudeatzaile estandarra erabiliz egiten da, eta beti ingurune birtual batean (1. gaian ikusitakoa), proiektuen mendekotasunak nahas ez daitezen:

```
pip install scikit-learn
```

Hurrengo adibideak ML-proiektu baten lan-fluxu klasikoa erakusten du, bost urrats argitan. Eskema honek ia edozein zeregin gainbegiraturako balio du: aldatu beharreko gauza bakarrak eredua eta, agian, aurreprozesatze-pausuak dira. Urrats bakoitzaren ordena errespetatzea funtsezkoa da: lehenik datuak banatu, eta gero entrenamendu-multzoarekin **soilik** doitu aurreprozesatzaileak, datu-isuria saihesteko.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
```

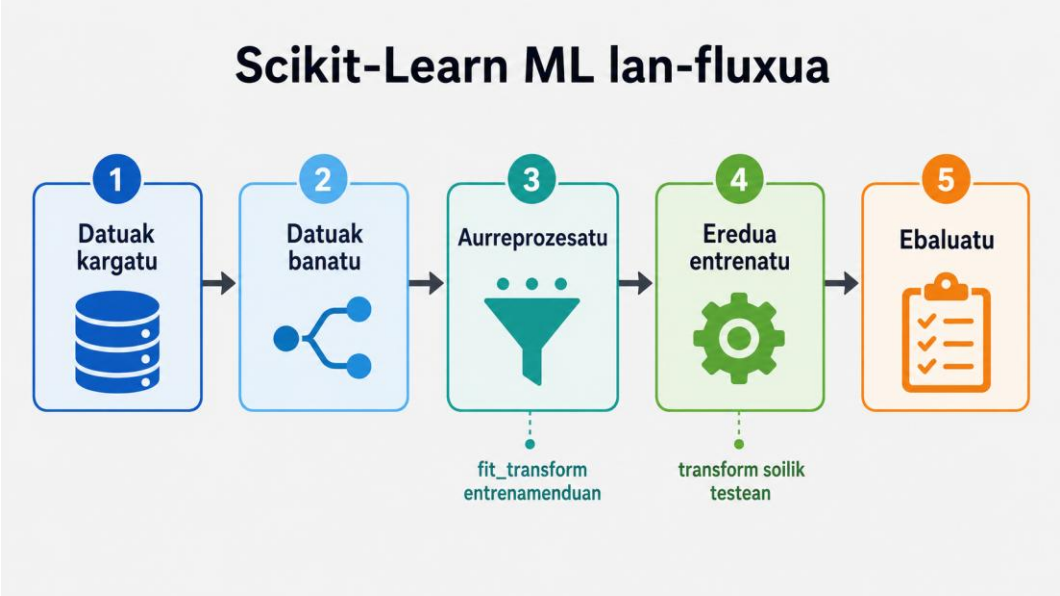
```
# 1. Datuak kargatu
X, y = load_iris(return_X_y=True)

# 2. Entrenamendu eta test multzoak banatu
X_entren, X_test, y_entren, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 3. Aurreprozesatu
eskala = StandardScaler()
X_entren = eskala.fit_transform(X_entren)   # fit + transform (entrenamendu datuetan)
X_test  = eskala.transform(X_test)          # transform soilik (test datuetan!)

# 4. Eredua entrenatu
eredua = KNeighborsClassifier(n_neighbors=5)
eredua.fit(X_entren, y_entren)

# 5. Iragarpenak egin eta ebaluatu
y_iragarkia = eredua.predict(X_test)
print(f"Zehaztasuna: {accuracy_score(y_test, y_iragarkia):.3f}")
```



1. Irudia: ML-proiektu baten bost urratsak Scikit-Learn-en — datuak kargatu, banatu, aurreprozesatu, entrenatu eta ebaluatu, `fit`/`transform` ordenarekin.

🔍 ***Adibidea: lehen eredua lan-postuan*** *Bezero baten datu-base bat jaso duzu: adina, antzinatasuna, kontratu-mota etab., eta zutabe bat esaten duena bezeroa joan egin den ala ez. Tutoreak esan dizu: "Probatu eredu bat ikusteko ea iragarri daitekeen nork alde egingo duen". Goiko bost urratseko eskema bera aplikatuko duzu: datuak banatu, eskalatu, KNN edo Random Forest entrenatu, eta zehaztasuna neurtu. Lehen emaitza ordu erdian lortzen da — eta hori da abiapuntua, ez amaiera.*

> 💡 ***Ideia nagusia*** *Scikit-Learn-en eredu guztiek interfaze bera dute: `fit()` ikasteko, `predict()` iragartzeko, `transform()` eraldatzeko. Behin patroi hau ikasita, ehunka eredu erabiltzen dakizu kode bera berrerabiliz.*
>
> ⚠ ***Kontuz: datu-isuria (data leakage)*** *Entrenamendua (`fit`) beti entrenamendu-datuekin egin behar da,* ***inoiz ez*** *test-datuekin. Bestela, test-datuetako informazioa "isuriko" litzateke ereduaren barrura, eta zure ebaluazioa engainagarria izango litzateke: laborategian primeran, baina errealitatean txarto. Akats hau oso ohikoa da hasiberrietan.*
>
> ☐ ***Ariketa 1.1 · 15 min · banaka*** ***Egoera****: Datu-multzo klasiko bat duzu eskura (`load_wine` edo `load_iris`).*
>
> ***Eginkizuna****: Idatzi goiko bost urratseko eskema osoa, baina `KNeighborsClassifier` eraldatuz `LogisticRegression` batera. Aldatu `test_size`-a 0.3-ra eta begiratu nola aldatzen den zehaztasuna.*
>
> ***Eztabaidatzeko****: Zergatik aplikatu behar zaie `transform` soilik test-datuei eta ez `fit_transform`? Zer gertatuko litzateke biei `fit_transform` egingo bagenie?*

1.2 Aurreprozesatzea eta Transformatzaileak

Aurreprozesatzea ML-proiektu ororen oinarrizko fasea da: ia inoiz ezin dira datu gordinak (*raw*) zuzenean ereduetara eraman. Praktikan, ML-ingeniari batek bere denboraren zatirik handiena fase honetan ematen du —askotan kodearen %70-80—. Atal honetan Scikit-Learn-en transformatzaile nagusiak ikusiko ditugu: zenbakizko aldagaien eskalatzea, kategorien kodeketa, balio faltadunen tratamendua, eta zutabe motak batera kudeatzeko `ColumnTransformer`.

ZENBAKIZKO ALDAGAIEN ESKALA-ALDAKETA

Eskala-aldaketa beharrezkoa da ezaugarrien arteko magnitude-aldea handia denean. Pentsa ezazu langile baten adina (20-65 tartean) eta soldata (20.000-70.000 tartean): zenbaki hutsetan, soldatak adina zanpatuko du distantziak kalkulatzean; esan nahi da, eskala berdinean ez egoteagatik batzuen eta besteen balioak alderatu ezinak dira muturrekoegiak izanagatik aldeak, eta, ondorioz, aztertu behar dituen makinak garrantzia kuantitaboa eman dakieke. Hots, bati bestearen gaineko pisua eman edo hutsaren pareko utzi iragarpenetarako ustezko balio makurrarengatik. Ordea, eskalatzeak denak maila berera ekartzen ditu. Distantzian oinarritutako ereduek (KNN) eta gradiente bidezko optimizazioa erabiltzen dutenek (erregresio logistikoa) bereziki onura ateratzen dute; zuhaitzetan oinarritutakoek (Random Forest), aldiz, ez dute eskalatzerik behar. Aukeraketa: `StandardScaler` da estandarra; `MinMaxScaler` goi/behe-mugak ezagunak direnean; `RobustScaler` outlier-ak daudenean.

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler
import numpy as np

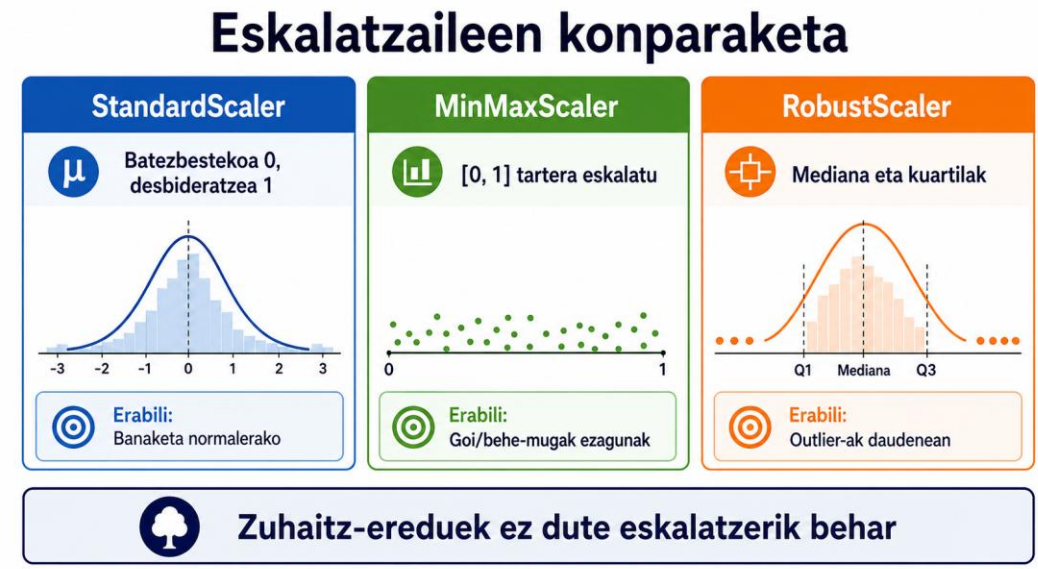
X = np.array([[1, 100], [2, 200], [3, 300], [4, 400], [5, 500]], dtype=float)

# StandardScaler: (x - batezbestekoa) / desbideratze tipikoa → batezbestekoa 0, desb. 1
std = StandardScaler()
```

```
print(std.fit_transform(X))

# MinMaxScaler: (x - min) / (max - min) → [0, 1] tartean
mm = MinMaxScaler()
print(mm.fit_transform(X))

# RobustScaler: mediana eta kuartilak → outlier-ei erresistente
rb = RobustScaler()
print(rb.fit_transform(X))
```



2. Irudia: StandardScaler, MinMaxScaler eta RobustScaler-en konparaketa — banaketa baten eraldaketa eta noiz erabili bakoitza.

KATEGORIA-ALDAGAIAK KODETU

Eredu gehienek zenbakizko sarrera behar dute. Kategoriadun aldagaiak —hiriak, departamentuak, produktu-motak— zenbakizko adierazpenera bihurtu behar dira lehenik. Hiru aukera nagusi: `LabelEncoder` helburu-aldagaia (y) kodetzeko; `OrdinalEncoder` aldagaiari berez ordena dagokionean ("txiki" < "ertain" < "handi"); eta `OneHotEncoder` ordenarik gabeko kategoriekin, kategoria bakoitzeko zutabe binario bat sortuz. Kontuz: kategoria askorekin (high cardinality), one-hot kodeketak dimentsionalitate handiegia sor dezake.

```
from sklearn.preprocessing import LabelEncoder, OneHotEncoder
import numpy as np

# LabelEncoder: helburuko aldagairako (y)
le = LabelEncoder()
y = ["katu", "txakur", "katu", "txori", "txakur"]
print(le.fit_transform(y))    # [0, 2, 0, 1, 2]
print(le.classes_)            # ['katu', 'txori', 'txakur']

# OneHotEncoder: ezaugarri-aldagaietarako (X)
ohe = OneHotEncoder(sparse_output=False, handle_unknown="ignore")
```

```
X_kat = np.array([["bilbo"], ["donostia"], ["gasteiz"], ["bilbo"]])
print(ohe.fit_transform(X_kat))
# [[1. 0. 0.] [0. 1. 0.] [0. 0. 1.] [1. 0. 0.]]
```

BALIO FALTADUNAK BETE (IMPUTATION)

Datu errealetan ohikoa da balio faltadunak (`NaN`) izatea: galdetegi-erantzun ezak, sentsoreen akatsak, datu-bilketako hutsuneak. Eredu gehienek ezin dute `NaN`-ekin lan egin. Bi aukera: errenkada osoa baztertu (informazio-galera handia) edo balio bat estimatuz bete (imputation). `SimpleImputer` da arruntena (batezbestekoa/mediana zenbakizkoetarako, moda kategorikoetarako). Funtsezkoa: imputatzailea entrenamendu-datuekin soilik doitu.

```
from sklearn.impute import SimpleImputer
import numpy as np

X = np.array([[1, 2, np.nan], [3, np.nan, 6], [np.nan, 8, 9], [4, 5, 6]])

imp = SimpleImputer(strategy="mean")   # "median", "most_frequent", "constant" ere
print(imp.fit_transform(X))
```

COLUMNTRANSFORMER: ZUTABE MOTA DESBERDINAK ALDI BEREAN

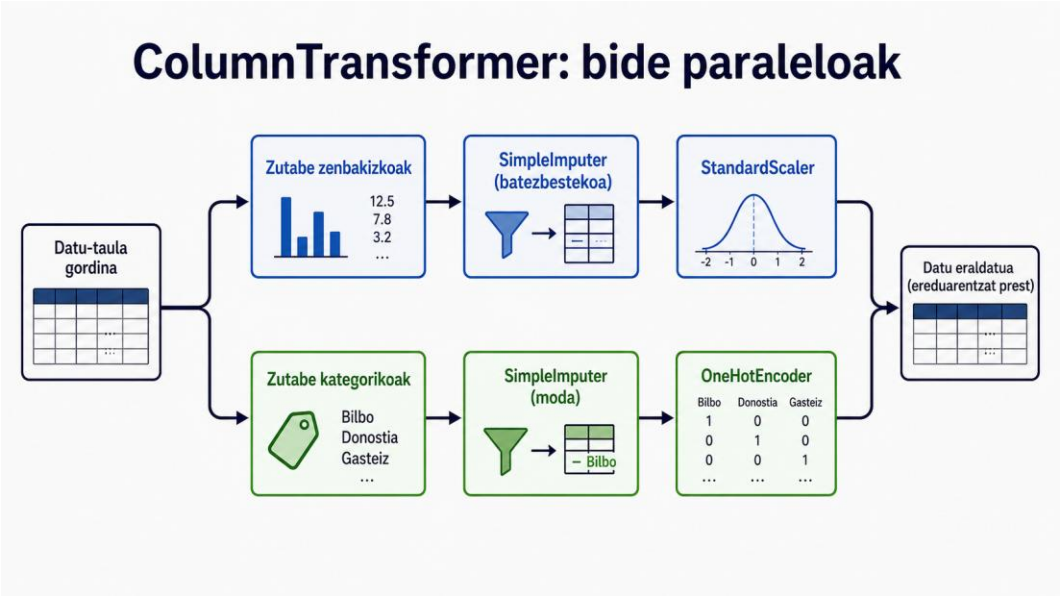
Datu-multzo errealetan, zutabe motak nahasten dira (zenbakizkoak, kategorikoak...). `ColumnTransformer` da aukerarik elegantena: zutabe-multzo bakoitzari bere transformazio-katea aplikatzen dio paraleloki eta emaitzak elkartzen ditu. Bi onura: kodea garbi mantentzen da eta pipeline orokor batean txerta daiteke gerora. Hori da, praktikan, profesionalki landutako ML-proiektu gehienen oinarrizko eskema.

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
import pandas as pd, numpy as np

df = pd.DataFrame({
    "adina":    [25, 30, np.nan, 35, 28],
    "lan_urte": [2, 5, 3, 7, 4],
    "hiria":    ["Bilbo", "Donostia", None, "Bilbo", "Gasteiz"],
})
zenbakizkoak = ["adina", "lan_urte"]
kategorikoak = ["hiria"]

aurreprozesatzailea = ColumnTransformer(transformers=[
    ("zenbak", Pipeline([
        ("bete",   SimpleImputer(strategy="mean")),
        ("eskala", StandardScaler())
    ]), zenbakizkoak),
    ("kateg", Pipeline([
        ("bete", SimpleImputer(strategy="most_frequent")),
        ("ohe",  OneHotEncoder(handle_unknown="ignore", sparse_output=False))
    ]), kategorikoak),
```

```
])
print(aurreprozesatzailea.fit_transform(df).shape)
```



3. Irudia: ColumnTransformer-ek zutabe zenbakizkoak eta kategorikoak bide paralelotan tratatzen ditu, eta gero emaitzak zutabe bakar batean elkartzen.

🔍 ***Adibidea: giza baliabideen taula garbitzea*** *Giza baliabideetatik Excel bat jaso duzu langileen datuekin: adina (zenbakizkoa, batzuk falta), departamentua (kategorikoa, batzuk hutsik), soldata (zenbakizkoa). Eredu batera eraman aurretik,* `ColumnTransformer` *batek aldi berean: zenbakizkoetan batezbestekoz betetzen eta eskalatzen ditu, eta kategorikoetan moda bidez betetzen eta one-hot kodetzen. Lerro gutxitan, taula nahasi bat ereduarentzat prest geratzen da.*

☐ ***Hausnartzeko*** *Zergatik da arriskutsua errenkada osoa baztertzea balio bat falta denean? Pentsa: 1.000 langileko datu-base batean, errenkada bakoitzak zutabe bat falta badu, zenbat errenkada geratuko litzaizkizuke? Eta informazio baliotsua galduko al zenuke?*

☐ ***Ariketa 1.2 · 20 min · taldeka*** ***Egoera****: Salmenta datubase bat duzue: produktua (kategorikoa), prezioa (zenbakizkoa), eskualdea (kategorikoa), stock (zenbakizkoa, batzuk falta).*

Eginkizuna*: Diseinatu* `ColumnTransformer` *bat zutabe bakoitzari tratamendu egokia emanez. Erabaki zer estrategia (mean/median/most_frequent) erabili stock faltadunarentzat eta arrazoitu.*

Eztabaidatzeko*: Eskualdea* `OneHotEncoder`*-ekin ala* `OrdinalEncoder`*-ekin kodetuko zenukete? Zergatik?*

Atal-laburpena: aurreprozesatzea ez da urrats hutsala: ereduaren errendimendua, neurri handi batean, bertan erabakitzen da.

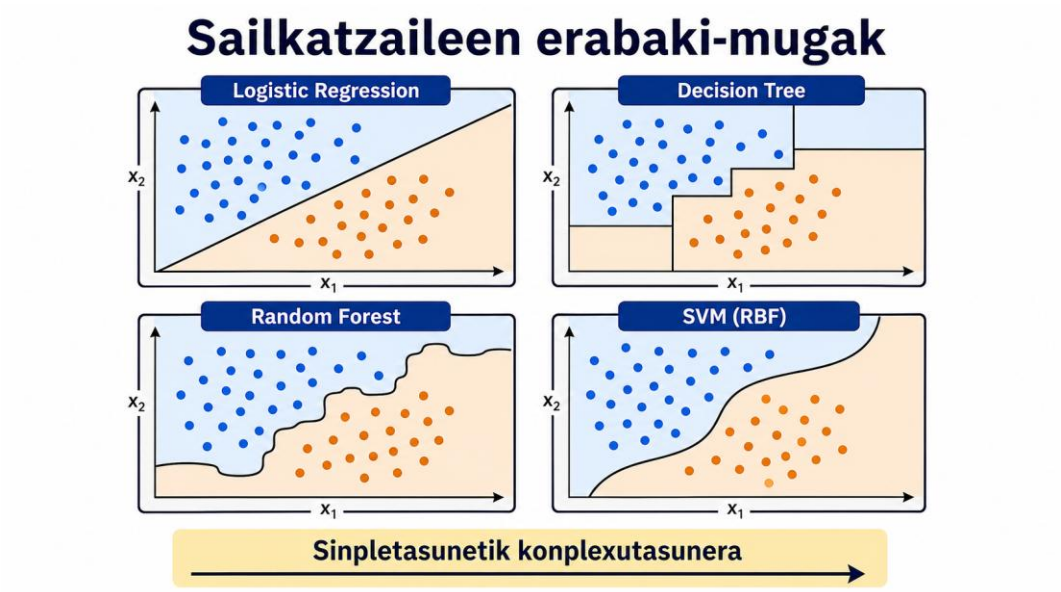
1.3 Sailkapen-ereduak

Sailkapen-arazoetan helburua **diskretua** da: lagin bati klase bat esleitzea (positiboa/negatiboa, gaixoa/osasuntsua, spam/ez-spam). Atal honetan industrian gehien erabiltzen diren sailkatzaileak ikusiko ditugu, sinpleenetik (Logistic Regression) konplexuagoetara (Gradient Boosting, SVM), bakoitzaren indarguneak aipatuz. Beti komeni da hainbat eredu probatzea oinarri-puntu (baseline) gisa: askotan eredu sinple batek emaitza onak ematen ditu eta interpretagarriagoa da.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC

# Datu-multzoa: bular-minbiziaren detekzioa (sailkapen binarioa)
X, y = load_breast_cancer(return_X_y=True)
X_ent, X_test, y_ent, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
eskala = StandardScaler()
X_ent = eskala.fit_transform(X_ent)
X_test = eskala.transform(X_test)

ereduak = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Random Forest":       RandomForestClassifier(n_estimators=100, random_state=42),
    "Gradient Boosting":   GradientBoostingClassifier(n_estimators=100, random_state=42),
    "SVM":                 SVC(kernel="rbf", probability=True, random_state=42),
}
for izena, eredua in ereduak.items():
    eredua.fit(X_ent, y_ent)
    print(f"\n{izena}:")
    print(classification_report(y_test, eredua.predict(X_test)))
```



4. Irudia: Lau sailkatzaileren erabaki-mugak (Logistic Regression, Decision Tree, Random Forest, SVM) datu-multzo berberaren gainean — sinpletasunetik konplexutasunera.

EBALUAZIO-METRIKAK: ZEHAZTASUNA EZ DA NAHIKOA

Metrika bakar batek ez du errealitate osoa erakusten. Klase desorekatuetan (adibidez, %1eko gaixotasun bat detektatzea) zehaztasuna soilik begiratzea oso engainagarria da: dena "osasuntsu" iragarriz, %99ko zehaztasuna lortuko genuke... gaixo bakar bat ere harrapatu gabe. Aurrerapen gisa: **doitasuna (precision)** garrantzitsua da positibo faltsuek kostu altua dutenean; **estaldura (recall)** kritikoa da negatibo faltsuek kostu altua dutenean (gaixotasunak ez detektatzea); **F1** bien arteko oreka da; **ROC-AUC** klase-banaketarekiko sendoa da.

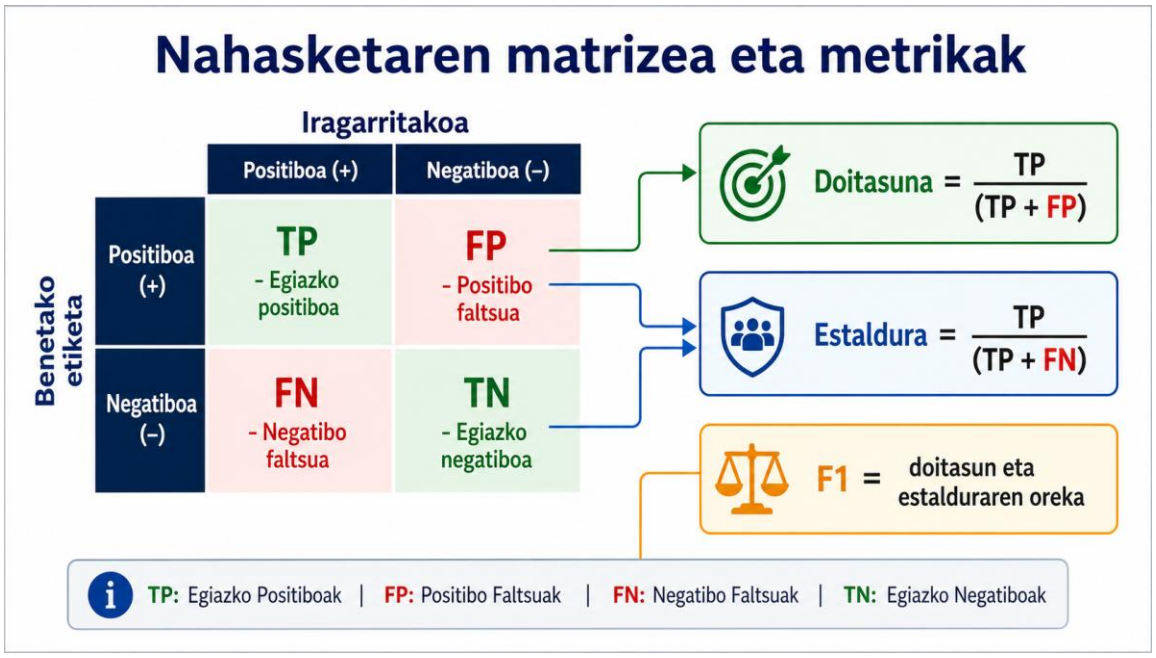
```
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, roc_auc_score, confusion_matrix)
import matplotlib.pyplot as plt
import seaborn as sns

rf = ereduak["Random Forest"]
y_irag  = rf.predict(X_test)
y_proba = rf.predict_proba(X_test)[:, 1]

print(f"Zehaztasuna:   {accuracy_score(y_test, y_irag):.4f}")
print(f"Doitasuna:     {precision_score(y_test, y_irag):.4f}")
print(f"Estaldura:     {recall_score(y_test, y_irag):.4f}")
print(f"F1 puntuazioa: {f1_score(y_test, y_irag):.4f}")
print(f"ROC-AUC:       {roc_auc_score(y_test, y_proba):.4f}")

# Nahasmen-matrizea
nahasket = confusion_matrix(y_test, y_irag)
fig, ax = plt.subplots(figsize=(6, 5))
sns.heatmap(nahasket, annot=True, fmt="d", cmap="Blues", ax=ax,
            xticklabels=["Negatiboa", "Positiboa"],
            yticklabels=["Negatiboa", "Positiboa"])
ax.set_ylabel("Benetako etiketa"); ax.set_xlabel("Iragarritako etiketa")
```

```
plt.tight_layout(); plt.show()
```



5. Irudia: Nahasketaren matrizea (TP, FP, FN, TN) eta nondik datozen doitasuna, estaldura eta F1 metrikak, intuizio bisualarekin.

Nahasmen-matrizeak ereduaren hutsegiteak (Positibo Faltsuak eta Negatibo Faltsuak) ikusten laguntzen digun arren, emaitza horiek metrika estandar eta konparagarri bihurtzeko **Sailkapen-txostena** (classification_report) erabiltzen da. Tresna honek klase bakoitzerako funtsezko hiru neurketa ateratzen ditu kolpe bakarrean (aurreko adibidean ez gisa, banaka bilatu behar direla):

```
from sklearn.metrics import classification_report

# Nahasmen-matrizearen osagarri gisa, metrika guztiak inprimatu
print("\nKlasifikazio-txostena:")
print(classification_report(y_test, y_iragarkia))
```

Metrika	Zer neurtzen du	Noiz da kritikoa
Zehaztasuna *edo ACCURACY*	Ondo iragarritako proportzio osoa	Klaseak orekatuta daudenean
Doitasuna *edo PRECISION*	Iragarritako positiboen artean benetakoak	Positibo faltsuak garestiak (spama)
Estaldura *edo RECALL*	Benetako positiboen artean harrapatutakoak	Negatibo faltsuak garestiak (gaixotasunak)
F1	Doitasun + estalduraren oreka	Klase desorekatuetan
ROC-AUC	Bereizteko gaitasun orokorra	Ereduen konparaketa orokorra

🔍 ***Adibide 1: spam-iragazkia vs minbizi-detekzioa*** *Bi kasu, bi metrika kritiko desberdin.* ***Spam-iragazki batean****, e-posta on bat spam gisa markatzea (positibo faltsua) larria da: bezeroak fakturak galduko ditu. Hortaz,* ***doitasuna*** *lehenetsiko genuke.* ***Minbizi-detekzioan****, gaixo bat osasuntsu gisa markatzea (negatibo faltsua) askoz*

larriagoa da. Hortaz, ***estaldura*** *lehenetsiko genuke. Metrika aukeratzea negozio-erabakia da, ez teknikoa soilik.*

🔍 ***Adibide 2:*** *Sute-alarma batean Precision-ek dio: "Alarma jotzen duenean, ba al dago suterik ala norbait ari al da erretzen tostadak?". Recall-ak erantzuten dio: "Benetan sutea badago, beti joko al du alarmak?"*

💡 ***Ideia nagusia*** *Klase-desorekarekin, zehaztasuna engainagarria da. F1 edo ROC-AUC erabili behar dira, eta metrika aukeratzea negozioaren testuingurutik dator: zer kostu du errore-mota bakoitzak?*

⚠ ***Kontuz: nahasketaren matrizea irakurtzen jakin*** *Hasiberri askok nahastu egiten dituzte positibo faltsua eta negatibo faltsua. Gomendio praktikoa: beti idatzi zer den "positiboa" zure kasuan (adibidez "tumorea gaiztoa da") eta gero galdetu "zer errore mota da arriskutsuena?". Hortik aterako duzu zein metrika lehenetsi.*

Atal-laburpena: sailkapen-arazoetan hainbat eredu probatu eta egokienak metrika anitzen arabera hautatu. Klase-desorekarekin kontuz: F1 edo ROC-AUC erabili ohiko zehaztasunaren ordez.

1.4 Erregresio-ereduak

Erregresio-arazoetan helburua **jarraitua** da: prezio bat iragartzea, tenperatura bat aurresatea, kontsumo bat estimatzea. Atal honetan erregresio-eredu nagusiak landuko ditugu (lineala, erregularizatuak eta zuhaitz-oinarritutakoak) eta metrika espezifikoak (MSE, RMSE, MAE, R^2) interpretatzen ikasiko dugu. Erregularizazioak (Ridge, Lasso) gainazterketa (overfitting) saihesten du, ezaugarrien artean korrelazio handia dagoenean bereziki.

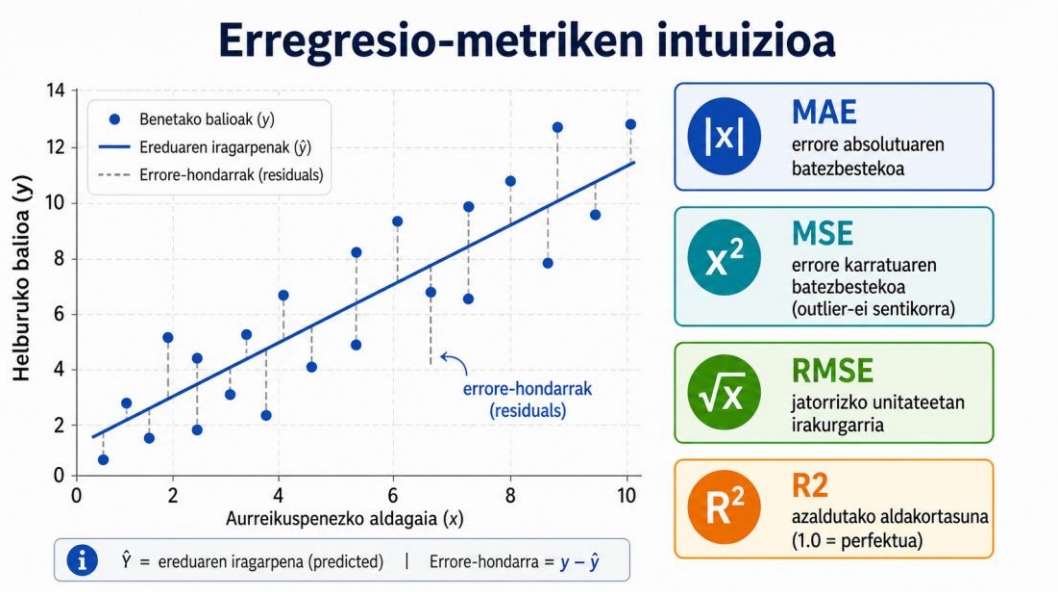
```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.ensemble import RandomForestRegressor
import numpy as np

X, y = load_diabetes(return_X_y=True)
X_ent, X_test, y_ent, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
eskala = StandardScaler()
X_ent = eskala.fit_transform(X_ent); X_test = eskala.transform(X_test)

def ebaluatu(izena, eredua):
    eredua.fit(X_ent, y_ent)
    y_irag = eredua.predict(X_test)
    print(f"\n{izena}:")
    print(f"  RMSE: {np.sqrt(mean_squared_error(y_test, y_irag)):.2f}")
    print(f"  MAE:  {mean_absolute_error(y_test, y_irag):.2f}")
    print(f"  R²:   {r2_score(y_test, y_irag):.4f}")
```

```
ebaluatu("Erregresio Lineala", LinearRegression())
ebaluatu("Ridge (L2)", Ridge(alpha=1.0))
ebaluatu("Lasso (L1)", Lasso(alpha=0.1))
ebaluatu("Random Forest", RandomForestRegressor(n_estimators=100, random_state=42))
```

Metriken irakurketa intuitiboa: **MSE** akatsen karratuaren batezbestekoa da eta akats handiak gogor zigortzen ditu (outlier-ei sentikorra); **RMSE** MSE-ren erro karratua da eta jatorrizko unitateetan irakurtzeko aukera ematen du (adibidez euroetan); **MAE** akats absolutuaren batezbestekoa da, outlier-ekiko egonkorragoa; **R²** ereduak datuen aldakortasunaren zer proportzio azaltzen duen adierazten du (1.0 = perfektua, 0 = batezbestekoa baino hobea ez).



6. Irudia: MSE, RMSE, MAE eta R²-ren intuizio bisuala — puntuen eta erregresio-lerroaren arteko distantziak (errore-hondarrak) nola bihurtzen diren metrika.

🔍 ***Adibidea: etxebizitzen prezioa iragartzen*** *Higiezin-enpresa batek etxeen prezioa iragartzeko eredua nahi du azalera, kokapena eta urteen arabera. RMSE 15.000 € ateratzen bada, esan nahi du, batez beste, ereduak 15.000 €-ko aldea egiten duela. Hori asko ala gutxi da? 300.000 €-ko etxeentzat onargarria izan daiteke; 90.000 €-koentzat, ez. Metrikak beti negozio-testuinguruan interpretatu behar dira.*

💡 ***Ideia nagusia*** *Erregresioan ia beti erregresio linealetik hasi (interpretagarria eta abiapuntu egokia), eta zuhaitzetan oinarritutakoetara igaro emaitzak nahikoak ez direnean. RMSE jatorrizko unitateetan irakur daiteke; R² portzentaje gisa.*

☐ ***Ariketa 1.4 · 15 min · banaka*** ***Eginkizuna****:* `load_diabetes` *datuekin, konparatu* `LinearRegression` *eta* `RandomForestRegressor`*-en R²-a. Gero, aldatu Ridge-ren* `alpha` *0.1, 1.0 eta 10.0 artean eta begiratu nola aldatzen den emaitza.* ***Eztabaidatzeko****: Zer esan nahi du R² negatibo batek? (Pista: batezbestekoa iragartzea baino okerragoa.)*

Atal-laburpena: erregresioan eredu lineal interpretagarrietatik hasi eta konplexuagoetara igaro behar da progresiboki. Metrikak (RMSE, MAE, R^2) konbinatuta interpretatu behar dira, beti negozio-testuinguruan.

1.5 Pipeline-ak

Pipeline-ak Scikit-Learn-en xarma profesionalaren benetako adierazpideak dira. Aurreprozesatze-pausuak eta eredua objektu bakar batean kateatzen dituzte, eta horrek bi onura nagusi ditu produkzio-mailan. Lehenik, datu-isuria sistemikoki saihesten dute: aurreprozesatzaileak entrenamendu-datuekin soilik doitzen dira `fit` deialdian. Bigarrenik, inferentzia-kodea drastikoki sinpletzen dute: `pipeline.predict(X_berriak)` egitea aski da, eta barnean pausu guztiak ordenan aplikatzen dira. Hori funtsezkoa da produkzio-zerbitzariekin integratzeko.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
import pandas as pd, numpy as np, joblib

# Datu sintetikoak (langile-txandaketa iragartzeko)
np.random.seed(42); n = 500
df = pd.DataFrame({
    "adina":         np.random.randint(22, 60, n),
    "soldata":       np.random.randint(20000, 70000, n),
    "lan_urte":      np.random.randint(0, 20, n),
    "departamentua": np.random.choice(["IT", "HR", "Finantza", "Eragiketak"], n),
    "txandakatze":   np.random.randint(0, 2, n)   # Helburua
})
df.loc[np.random.choice(df.index, 30), "adina"] = np.nan

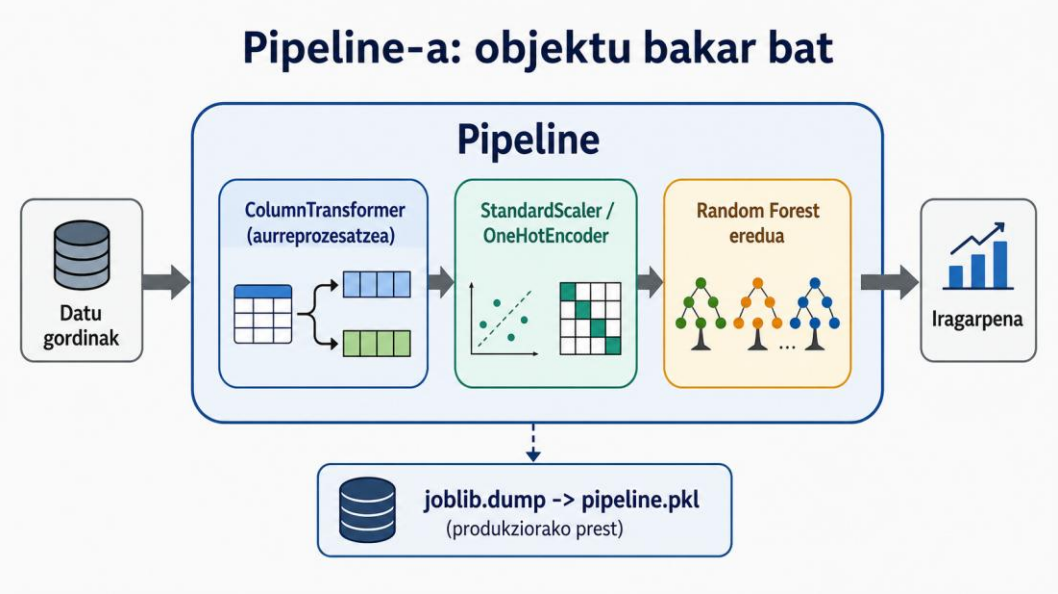
X = df.drop("txandakatze", axis=1); y = df["txandakatze"]
X_ent, X_test, y_ent, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

aurreproz = ColumnTransformer([
    ("zenbak", Pipeline([("bete", SimpleImputer(strategy="mean")),
                         ("eskala", StandardScaler())]),
     ["adina", "soldata", "lan_urte"]),
    ("kateg", Pipeline([("bete", SimpleImputer(strategy="most_frequent")),
                        ("ohe", OneHotEncoder(handle_unknown="ignore",
sparse_output=False))]),
     ["departamentua"]),
])

# Pipeline osoa: aurreprozesatzailea + eredua objektu BAKAR batean
pipeline = Pipeline([
    ("aurreproz", aurreproz),
    ("eredua",    RandomForestClassifier(n_estimators=100, random_state=42))
```

```
])
pipeline.fit(X_ent, y_ent)    # dena batera entrenatu
print(classification_report(y_test, pipeline.predict(X_test)))

# Pipeline OSOA gorde eta kargatu (produkziorako prest)
joblib.dump(pipeline, "txandakatze_pipeline.pkl")
berreskuratua = joblib.load("txandakatze_pipeline.pkl")
print(berreskuratua.predict(X_test[:5]))
```



7. Irudia: Pipeline batek aurreprozesatzea eta eredua objektu bakar batean biltzen ditu — `fit`, `predict` eta `joblib` bidezko gordeketa, datu gordinetatik iragarpenera.

🔍 ***Adibidea: produkziora bidaltzen den eredua*** *Ereduak laborategian funtzionatzen du, baina produkziora bidali behar duzu. Pipeline gabe, web-zerbitzariak datu berriak eskuz eskalatu, kategoriak kodetu eta gero ereduari pasa beharko lizkioke — eta edozein akats datu-isuria edo errore bat sortuko luke. Pipeline batekin, `joblib.load("pipeline.pkl")` eta `pipeline.predict(datu_berriak)` aski dira: aurreprozesatze osoa barnean dator. Hori da produkziorako prest dagoen kodea.*

💡 ***Ideia nagusia*** *Pipeline-ek aurreprozesatzea eta eredua bateratzen dituzte: datu-isuria saihesten dute, kodea garbitzen, eta inferentzia produkziora eramatea erraztu egiten dute. ML-proiektu seriotan pipeline gabe lan egitea akats handitzat hartzen da.*

☐ ***Ariketa 1.5 · 25 min · taldeka*** ***Egoera****: Aurreko ariketako datu-base bat duzue zutabe nahasiekin.*

Eginkizuna*: Eraiki pipeline oso bat (ColumnTransformer + eredua), entrenatu, gorde `joblib`-ekin, eta beste script batean kargatu eta iragarpen bat egin. Egiaztatu emaitza berdina dela.*

***Eztabaidatzeko**: Zer abantaila ditu pipeline-a gordetzeak eredua bakarrik gordetzearen aldean?*

Atal-laburpena: pipeline-ek aurreprozesatzea eta eredua objektu bakar batean lotzen dituzte. Horrek datu-isuria saihesten du, kodea garbitzen, eta inferentzia produkziora eramatea erraztu egiten du.

1.6 Ebaluazioa eta hiperparametroen doiketa

☐ ***Errepasoa: 5072 moduluan ikusi duzue*** *Balioztatze-gurutzatuaren oinarriak (K-Fold, Stratified K-Fold, train/val/test banaketa, data leakage)* ***5072 moduluan*** *landu dira sakon (`5072_3_Balidazio_Metodologia`). Sailkapen- eta erregresio-metrikak (accuracy, precision, recall, F1, ROC-AUC, MSE/RMSE/MAE/R²) `5072_2_Ikasketa_Gainbegiratua`-n. Hemen scikit-learn-en API-an zentratzen gara: nola kalkulatu metrikak `cross_val_score`-rekin eta hiperparametroak nola optimizatu `GridSearchCV`/`RandomizedSearchCV`-rekin.*

Eredu bat entrenatuta, bi galderari erantzun behar zaie: zenbat fida gaitezke estimatutako errendimenduan, eta nola hobetu hiperparametroak doituz? Atal honetan **balioztatze-gurutzatua** (cross-validation) eta hiperparametroen bilaketa sistematikoa (**GridSearchCV** eta **RandomizedSearchCV**) lantzen ditugu. Tresna hauek ad-hoc esperimentazioa metodo profesional eta erreproduzigarri batera aldatzen dute.

BALIOZTATZE-GURUTZATUA (CROSS-VALIDATION)

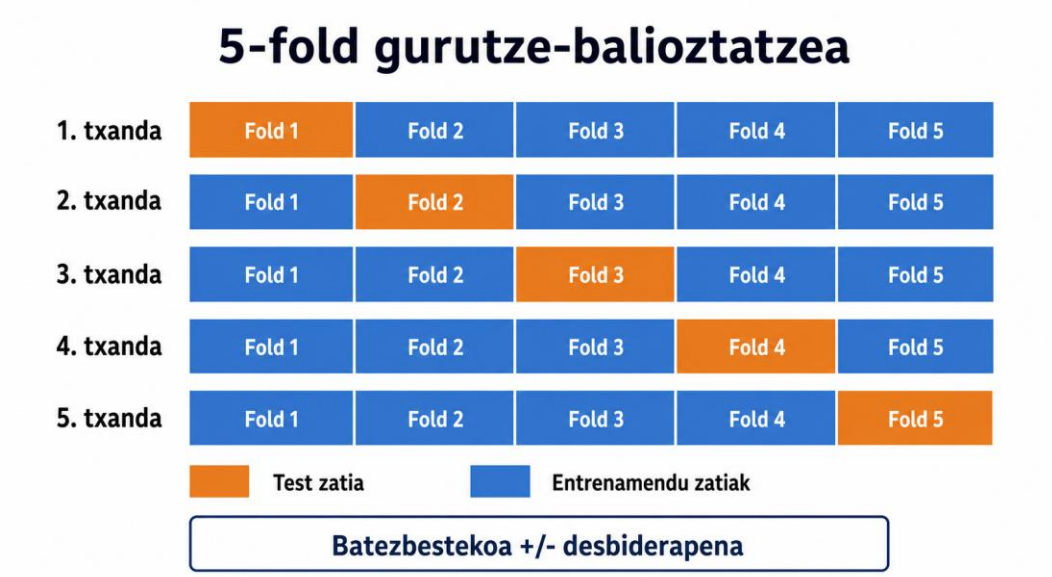
Entrenamendu/test banaketa bakar bat ausazkoa da: agian zorionez (edo zoritxarrez) banatu da. Balioztatze-gurutzatuak hori konpontzen du: datuak *k* zatitan banatzen ditu eta *k* aldiz entrenatzen/ebaluatzen du, txanda bakoitzean zati ezberdin bat test gisa erabiliz. Horrela errendimenduaren estimazio sendoagoa lortzen da, batezbestekoarekin eta desbiderapenarekin batera. `StratifiedKFold` funtsezkoa da sailkapenean, klaseen proportzioa mantentzeko.

```
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_breast_cancer

X, y = load_breast_cancer(return_X_y=True)
rf = RandomForestClassifier(n_estimators=100, random_state=42)

puntuazioak = cross_val_score(rf, X, y, cv=5, scoring="roc_auc")
print(f"ROC-AUC (5-fold): {puntuazioak.mean():.4f} ± {puntuazioak.std():.4f}")

skf = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)
print(f"F1 (10-fold): {cross_val_score(rf, X, y, cv=skf, scoring='f1').mean():.4f}")
```



8. Irudia: 5-fold Balioztatze-gurutzatuaren eskema — datuak bost zatitan, eta txanda bakoitzean zati bat test gisa eta gainerakoak entrenamendurako.

GRIDSEARCHCV ETA RANDOMIZEDSEARCHCV

`GridSearchCV`-k zerrendatutako konbinazio guztiak probatzen ditu sistematikoki, Balioztatze-gurutzatuarekin. Abantaila: espazio osoa estaltzen du. Desabantaila: konbinazioak azkar hazten dira, esponentzialki. `RandomizedSearchCV`-k ausazko konbinazioak laginatzen ditu (`n_iter` parametroaz kontrolatuta): azkarragoa eta sarritan eraginkorragoa parametro asko daudenean.

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier

parametro_sarea = {
    "n_estimators":      [50, 100, 200],
    "max_depth":         [None, 5, 10, 20],
    "min_samples_split": [2, 5, 10],
}
rf = RandomForestClassifier(random_state=42)
bilaketa = GridSearchCV(rf, parametro_sarea, cv=5, scoring="roc_auc", n_jobs=-1, verbose=1)
bilaketa.fit(X, y)

print(f"Parametro onenak: {bilaketa.best_params_}")
print(f"ROC-AUC onena:    {bilaketa.best_score_:.4f}")
eredu_onena = bilaketa.best_estimator_   # parametro onenekin prest
```

> 💡 ***Ideia nagusia*** *Balioztatze-gurutzatuak eta hiperparametro-bilaketa sistematikoak bereizten dute ML-proiektu seriotik amateurra. Beti erreportatu emaitzak desbiderapen-tartearekin (`X ± Y`).*

> ⚠ ***Kontuz: GridSearch garestia izan daiteke*** *3 × 4 × 3 = 36 konbinazio × 5 fold = 180 entrenamendu. Parametro gehiago gehitzen badituzu, esponentzialki hazten da eta orduak*

iraun dezake. Espazio handietan, `RandomizedSearchCV` *(edo Optuna bezalako bilaketa Bayestarra) gomendatzen da.*

☐ ***Ariketa 1.6 · 25 min · taldeka Eginkizuna****: Aukeratu eredu bat eta egin* `GridSearchCV` *bat 2-3 hiperparametrorekin. Gero,* `RandomizedSearchCV` *batekin antzeko espazioa estali* `n_iter=20`*-rekin. Konparatu: zein da azkarragoa? Antzeko emaitza lortzen al dute?*

Eztabaidatzeko*: Noiz konpentsatzen du GridSearch erabiltzea RandomizedSearch-en ordez?*

Atal-laburpena: Balioztatze-gurutzatua eta hiperparametro-bilaketa sistematikoa funtsezko praktika profesionalak dira. Emaitzak beti desbiderapen-tartearekin erreportatu.

1.7. Atalase-mugaren doiketa (Threshold tuning)

Klasifikazio-algoritmo gehienek (Random Forest, Logistic Regression, etab.) atzealdean probabilitate bat kalkulatzen dute lehenik. Scikit-Learn-en `predict()` metodoa erabiltzen dugunean, liburutegiak automatikoki % 50eko atalasea (0.5) aplikatzen du: probabilitatea > 0.5 bada, klase positibotzat joko du.

Muga zurrun hori ondo dago klaseak orekatuta daudenean. Baina mundu errealeko arazo askotan (non hutsegite baten prezioa izugarria den), % 50eko muga hori arriskutsuegia da. Horrelakoetan, erabaki itsua hartu beharrean, `predict_proba()` metodoa erabiltzen da probabilitateak zuzenean lortzeko, eta gure negoziorako egokiena den atalasea eskuz aplikatzen da.

🔍 ***Adibidea: Diagnostiko Medikoa eta "Geldialdi Aseguratuaren Paradoxa"***
Demagun gaixotasun larri bat aurreikusten duen AA eredu bat dugula ospitale batean. Analisiak eginda, ereduak gaixotasuna garatzeko %35eko probabilitatea dagoela kalkulatu du.
Berezko `predict()` *metodoa erabiliz gero (lehentasunezko 0.5eko muga finkoduna), ereduak gaixoa "Osasuntsu" dagoela esango luke (0.35 < 0.5). Medikuak ez luke froga gehiagorik eskatuko, eta gaixoa egoera kritikoan itzuliko litzateke. Industrian eta negozioetan, portaera honi "Geldialdi Aseguratuaren Paradoxa" esaten zaio: matematika zorrotzaren ikuspuntutik %50era iritsi ez delako ezkutatzen den arriskuak, epe luzera, erabateko hondamendia bermatzen baitu.*

Eztabaidatzeko*: Jolastuko zinateke errusiar erruletara, bala kopurua erdiak baino gutxiago dela bakarrik bermatuko balizute? Edo, beste era batera esanda: erdiak baino gehiago balak izan arte jarraituko zenuke?*

Arazoa konpontzeko, probabilitateak atera eta atalasea (adibidez 0.15era) jaitsi behar da. Horrela, Recall-a igoko dugu (arrisku guztiak harrapatuz), nahiz eta pertsona osasuntsu batzuei alferrikako

probak egitea onartu (Precision jaitsiz). **Atalase egokiena zein den erabakitzeko, ezinbestekoa da Precision-Recall kurba marraztea.**

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import PrecisionRecallDisplay, classification_report

# Datu-multzo desorekatua simulatu (adib. gaixotasun arraro baten diagnostikoa: % 5 gaixo)
X, y = make_classification(n_samples=2000, n_features=10, n_classes=2,
                           weights=[0.95, 0.05], random_state=42)
X_ent, X_test, y_ent, y_test = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y)

eredua = RandomForestClassifier(class_weight='balanced', random_state=42)
eredua.fit(X_ent, y_ent)

# 1. 'predict()' erabili beharrean, probabilitateak atera
# Bigarren zutabea [:, 1] klase positiboaren probabilitatea da
y_prob = eredua.predict_proba(X_test)[:, 1]

# 2. Precision-Recall kurba marraztu
fig, ax = plt.subplots(figsize=(7, 5))
PrecisionRecallDisplay.from_estimator(eredua, X_test, y_test, ax=ax)
ax.set_title("Precision-Recall Kurba (Datu Desorekatuak)")
plt.show()

# Kurba ikusita, arrisku guztiak harrapatzeko (Recall altua) atalasea 0.15era jaitsi behar
dugu.

# 3. Gure negoziorako atalase seguru bat ezarri
ATALASEA = 0.15

# 4. Probabilitatea atalasea baino handiagoa denean 1 itzuli, bestela 0
y_iragarkia_doitua = (y_prob > ATALASEA).astype(int)

# 5. Ebaluatu metrikak berriro, orain Negatibo Faltsu askoz gutxiagorekin!
print(classification_report(y_test, y_iragarkia_doitua))
```

> 💡 ***Ideia nagusia:*** *Ez utzi Scikit-Learni zure negozioaren erabaki kritikoak itsu-itsuan hartzen. Matxura, iruzurrak edo gaixotasunak detektatzeko `predict_proba()` erabili ordea, atalasea arrisku-mailara igotzeko.*

2. Hugging Face: Eredu Aurreentrenatuen Ekosistema

Hugging Face-k aldatu egin du adimen artifizialaren paisaia: aurreentrenatutako milaka eredu jartzen ditu eskura interfaze uniforme batekin. Atal honen helburua bikoitza da: alde batetik eredu aurreentrenatuen filosofia ulertzea —milioika dokumenturekin entrenatutako sare handi bat datu gutxirekin proiektu espezifiko batera egokitzea—, eta bestetik `pipeline`, `transformers`, `tokenizers` eta `datasets` liburutegiak praktikan menderatzea. NLPan, ikusmenean eta audioan, Hugging Face *de facto* estandarra bihurtu da.

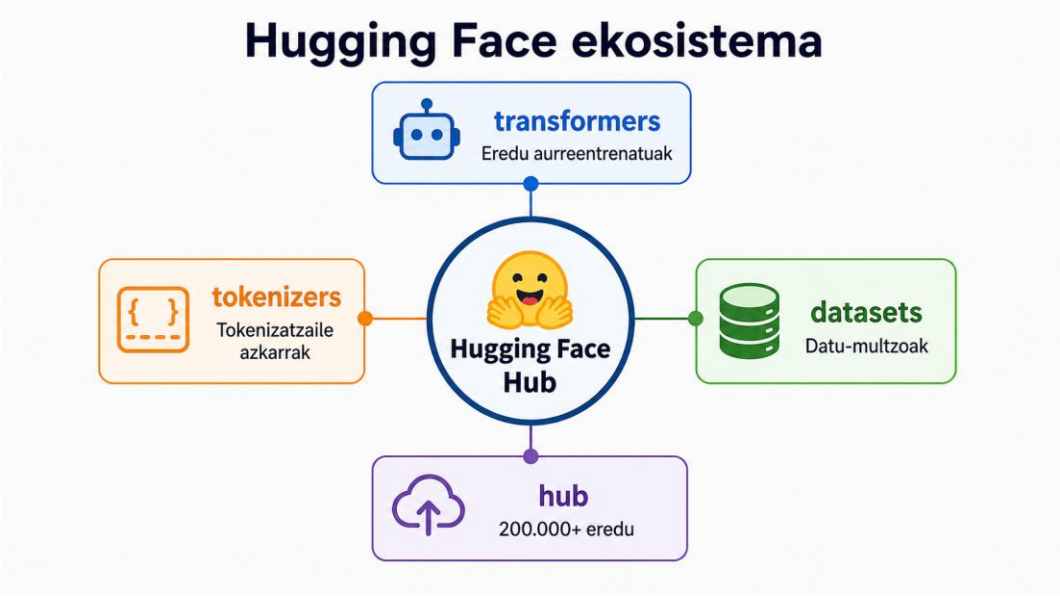
2.1 Hugging Face ekosistema

☐ ***Errepasoa: 5071 moduluan ikusi duzue*** *Hugging Face-ren ekosistema kontzeptuala, hizkuntza-eredu txikiak (SLMs), prompt engineering, ChatGPT/Gemini/Claude eredu nagusiak, NotebookLM eta kuantizazioa* ***5071 moduluan*** *landu dira (`5071/01_NLP`). Eredu sortzaileen aldetik (Stable Diffusion, DCGAN) 5072-ko `03_AI_Sortzailea` ataleko Python ariketan (`03_sortzailea_notebook_GenAI.ipynb`) praktikatu dituzue. Hemen Hugging Face* ***Python liburutegi gisa*** *sakontzen dugu: instalazioa, `pipeline()` API-a, eta FastAPI eta Streamlit aplikazioetan integrazioa.*

Hugging Face NLP eta beste IA zereginen ekosistema nagusia da. Bere ideia nagusia eredu aurreentrenatuena da: milioika testu-dokumenturekin entrenatutako eredu erraldoiek hizkuntza-jakintza orokorra eskuratzen dute. Gero, datu gutxirekin zeregin espezifikoetara egokitu daitezke (fine-tuning), edo zuzenean erabili. Pentsa ezazu eredu bat unibertsitate osoa ikasi duen ikasle bat bezala: ez duzu zerotik irakatsi behar, baizik eta zure enpresaren berezitasunak soilik.

Osagaia	Deskribapena
`transformers`	Eredu aurreentrenatuen liburutegia (BERT, GPT, T5...)
`datasets`	Datu-multzo ezagunen bilduma
`tokenizers`	Tokenizatzaile azkarrak
`hub`	Ereduen biltegi publikoa (200.000+ eredu)

```
pip install transformers datasets tokenizers
```



9. Irudia: Hugging Face-ren lau osagai nagusiak (transformers, datasets, tokenizers, hub) eta haien arteko harremana ekosistema gisa.

> 💡 ***Ideia nagusia*** *Eredu aurreentrenatuak ikasketak bukatu dituen langile bat bezalakoak dira: ez dituzu zerotik trebatu behar. Hizkuntza orokorra badakite; zuk soilik zure kasura egokitu behar dituzu.*
>
> ☐ ***Ariketa 2.1 · 10 min · banaka*** ***Eginkizuna****: Sartu (https://huggingface.co/models) gunean. Bilatu "euskara" hizkuntza-iragazkiarekin. Zenbat eredu daude? Aukeratu bat eta deskribatu zertarako balio duen.*
>
> ***Eztabaidatzeko****: Zergatik da garrantzitsua eredu aurreentrenatuak partekatzea komunitatean? Zer aurrezten du enpresa batek?*

2.2 Pipeline API: inferentzia azkarra

`pipeline` funtzioa Hugging Face-ren abstrakzio mailarik altuena da: zeregin baten izena ematea aski da (`"sentiment-analysis"`, `"summarization"`...), eta eredu egokia automatikoki deskargatu, kargatu eta inferentziarako prest jartzen du. Hiru lerrotan, prototipo funtzional bat duzu. Atal honetan zeregin nagusiak ikusiko ditugu.

```
from transformers import pipeline

# ---- Sentimenduen analisia ----
sailkatzailea = pipeline("sentiment-analysis")
emaitzak = sailkatzailea([
    "I love this product, it's amazing!",
    "This is the worst experience I have ever had.",
])
for e in emaitzak:
```

```
    print(f"Etiketa: {e['label']}, Konfiantza: {e['score']:.4f}")
# POSITIVE 0.9998 / NEGATIVE 0.9994

# ---- Galdera-erantzun sistema (extractive QA) ----
qna = pipeline("question-answering")
testuingurua = """Scikit-learn Python-en ikasketa automatikoaren liburutegi nagusia da.
2007an David Cournapeau-k sortu zuen."""
erantzuna = qna(question="Noiz sortu zen Scikit-learn?", context=testuingurua)
print(f"{erantzuna['answer']} (konfiantza: {erantzuna['score']:.4f})")

# ---- Testu-laburpena (abstractive) ----
laburtzailea = pipeline("summarization", model="facebook/bart-large-cnn")
print(laburtzailea("Deep learning is a subset of machine learning that uses "
                   "neural networks with many layers...", max_length=40, min_length=10))

# ---- Irudi-sailkapena (Vision Transformer) ----
irudi_sailk = pipeline("image-classification", model="google/vit-base-patch16-224")
```



10. Irudia: Hugging Face `pipeline` APIaren zeregin nagusiak (sentiment-analysis, QA, summarization, translation, image-classification) eta bakoitzaren sarrera/irteera.

🔍 ***Adibidea: 10.000 iruzkin minutu gutxitan*** *Sarrerako eszenara itzul gaitezen: marketin-taldeak milaka bezero-iruzkin ditu.* `pipeline("sentiment-analysis")` *batekin, iruzkin guztiak banan-banan sailka ditzakezu (positibo/negatibo) eta konfiantza-balio batekin. Begizta sinple batean, milaka iruzkin minututan prozesatzen dira, eta emaitza taula batean biltzen da. Eskuz astebete behar zena, kodez egun erdian.*

💡 ***Ideia nagusia*** `pipeline` *API-a Hugging Face-rekin lan egiteko biderik azkarrena da. Prototipaketa-fasean kasu gehienak estaltzen ditu, eta askotan ekoizpenerako ere nahikoa izaten da, mekanika-maila baxuagora jaitsi gabe.*

⚠ ***Kontuz: lehen deia motela da*** *`pipeline`-k lehen aldian eredua internetetik deskargatzen du (ehunka MB izan daitezke). Lehen exekuzioa motela izango da; ondorengoak azkarrak (cachean dago). Produkzioan, eredua aurrez deskargatu eta lokalki gorde behar da.*

☐ ***Ariketa 2.2 · 20 min · taldeka*** ***Egoera****: Hotel-kate batek TripAdvisor-eko iruzkinak aztertu nahi ditu.* ***Eginkizuna****: Idatzi 8-10 iruzkin fikziozko (ingelesez), pasa `pipeline("sentiment-analysis")`-tik eta sortu laburpen-taula bat: zenbat positibo, zenbat negatibo, eta zein konfiantza-maila.* ***Eztabaidatzeko****: Zer egingo zenukete konfiantza baxuko iruzkinekin (adibidez 0.55)? Fida al gintezke?*

2.3 Tokenizatzaileak eta ereduak

Tokenizatzaileak Transformer-eredu ororen lehen kapa dira: testua zenbaki-sekuentzia bihurtzen dute, eredu neuronalak prozesa dezakeen formatuan. BERT eta GPT-ek WordPiece edo BPE bezalako *subword* tokenizatzaileak erabiltzen dituzte: horrek hiztegitik kanpoko hitzak (OOV) kudeatzeko aukera ematen du, hitzak zati txikiagotan banatuz. Hori bereziki erabilgarria da hizkuntza anitzeko ereduentzat.

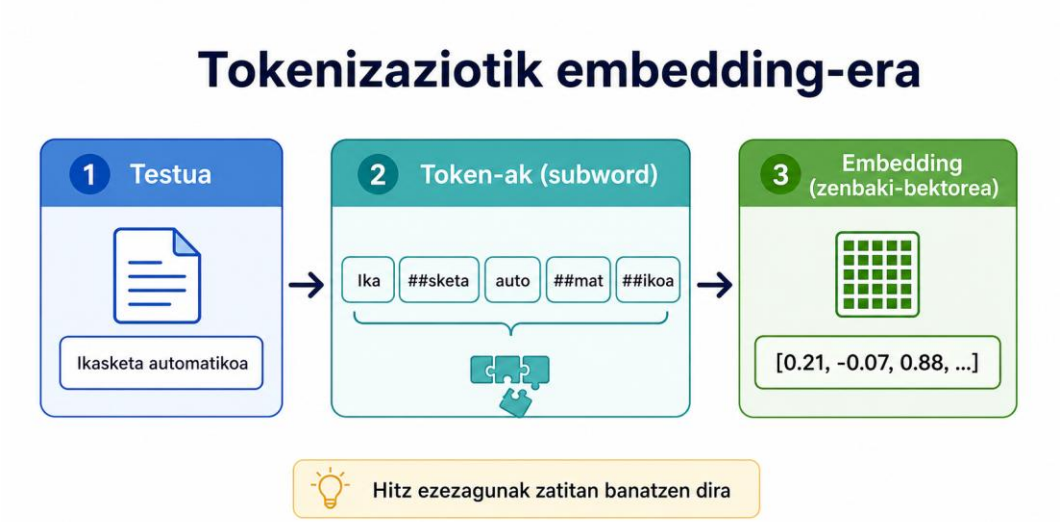
```
from transformers import AutoTokenizer, AutoModel
import torch

eredu_izena = "bert-base-multilingual-cased"   # euskara ere ulertzen du
tokenizatzailea = AutoTokenizer.from_pretrained(eredu_izena)
eredua = AutoModel.from_pretrained(eredu_izena)

testua = "Ikasketa automatikoa oso interesgarria da."
tokenak = tokenizatzailea(testua, return_tensors="pt", padding=True, truncation=True)

print(tokenizatzailea.convert_ids_to_tokens(tokenak["input_ids"][0]))
# ['[CLS]', 'Ika', '##sketa', 'auto', '##mat', '##ikoa', 'oso', ...]

with torch.no_grad():
    irteera = eredua(**tokenak)
embedding = irteera.last_hidden_state[:, 0, :]   # CLS tokena = testu osoaren embedding
print(f"Embedding forma: {embedding.shape}")     # [1, 768]
```



11. Irudia: Testu batetik token ID-etara eta embedding-etara — "Ikasketa automatikoa" hitza subword zatitan banatuta eta zenbaki-bektore bihurtuta.

> 🔍 ***Adibidea: zergatik banatzen den hitza zatitan*** *"Ikasketa" hitza ez dago BERT-en hiztegian bere horretan, baina "Ika" eta "##sketa" zatiak bai. Horrela, ereduak sekula ikusi ez duen hitz bat ere kudea dezake, ezagutzen dituen zatien bidez. Hori da subword tokenizazioaren bertutea: hiztegi mugatu batekin, edozein hitz adieraz daiteke, baita asmatutakoak ere.*
>
> 💡 ***Ideia nagusia*** *Tokenizatzaileek testua zenbaki bihurtzen dute. Subword tokenizazioak (WordPiece, BPE) hitz ezezagunak zatitan banatuz kudeatzen ditu, hiztegi mugatu batekin edozein testu adierazteko aukera emanez.*
>
> ☐ ***Ariketa 2.3 · 15 min · banaka*** ***Eginkizuna****: Tokenizatu hiru esaldi euskaraz* `bert-base-multilingual-cased`*-ekin eta begiratu nola banatzen diren hitzak* `##` *markarekin. Zein hitz banatzen dira gehien? Zergatik uste duzu?* ***Eztabaidatzeko****: Zergatik kudeatzen dira euskararen hitzak hain zatituta ingelesarenak baino gehiago? (Pista: hiztegia gehienbat ingelesez entrenatuta dago.)*

2.4 Datasets liburutegia

`datasets` liburutegia Hugging Face-ren beste pieza nagusietakoa da: milaka datu-multzo lerro bakar batekin kargatzeko aukera ematen du, eta memoria-eraginkortasunerako optimizatuta dago (Apache Arrow erabiltzen du barnean, datu-multzo erraldoiak diskotik laginatzeko). API funtzionala eskaintzen du (`map`, `filter`, `select`, `shuffle`) eta Pandas-ekin elkarreragingarritasuna bermatzen du.

```
from datasets import load_dataset, Dataset
import pandas as pd
```

```
# Datu-multzo bat kargatu (lehen kargatzean deskargatu, gero cachean)
datu_multzoa = load_dataset("imdb")
print(datu_multzoa)
# train: 25000 errenkada, test: 25000 errenkada

entren = datu_multzoa["train"]
print(entren[0])                # {'text': '...', 'label': 1}

# Pandas-era bihurtu
df = entren.to_pandas()

# Filtrak eta transformazioak
positiboak = entren.filter(lambda x: x["label"] == 1)
print(f"Iritzi positiboak: {len(positiboak)}")

# Pandas DataFrame batetik Dataset sortu
df_propio = pd.DataFrame({"testua": ["Adibide bat", "Beste bat"], "etiketa": [0, 1]})
datu_propioa = Dataset.from_pandas(df_propio)
```

🔍 ***Adibidea: datu-multzo erraldoi bat memoria betetzeke*** *Iruzkin-datu-base bat 50 GB-koa da, eta zure ordenagailuak 16 GB RAM bakarrik ditu. Pandas-ekin kargatzea ezinezkoa litzateke.* `datasets`*-ek, ordea, Apache Arrow erabiliz, diskotik laginatzen ditu datuak behar diren unean — memoria betetzeke. Horregatik da hain erabilia eredu handiak entrenatzeko datu-prestaketan.*

💡 ***Ideia nagusia*** `datasets` *liburutegiak datu-multzo erraldoiak memoria-eraginkortasunez kudeatzen ditu eta API funtzional garbia eskaintzen du. Pandas-ekin bateragarria da, baina manipulazio konplexuetan* `datasets`*-en bertan egitea hobea da errendimenduagatik.*

☐ ***Ariketa 2.4 · 15 min · taldeka***

Eginkizuna*: Kargatu* `imdb` *datu-multzoa, filtratu iritzi positiboak, eta erabili* `select` *lehen 100 lagin hartzeko. Bihurtu Pandas-era eta erakutsi lehen 5 errenkadak.*

Eztabaidatzeko*: Noiz erabiliko zenukete* `datasets` *eta noiz Pandas? Zer abantaila du bakoitzak?*

Atal-laburpena: Hugging Face ekosistemak eredu aurreentrenatuak, datu-multzoak eta tokenizatzaileak interfaze uniforme batean biltzen ditu. `pipeline` API-arekin minutu gutxitan prototipo funtzionalak eraiki daitezke NLP, ikusmen eta audio zereginetarako.

3. FastAPI: REST API-ak Python-ekin

Atal honetan REST API-ak Python-ekin nola eraiki ikasiko dugu, FastAPI framework-a erabiliz. Helburua da ML eredu bat edo edozein logika web-zerbitzu baten bidez modu sendoan eskaintzeko gaitasuna lortzea: ibilbideen definizioa, sarreren balioztatzea, dokumentazio automatikoa eta segurtasun-mekanismoak. FastAPI gaur egun industrian gehien erabiltzen den Python framework-etako bat da, eta MLOps eta GenAI inguruneetako funtsezko pieza bihurtu da.

3.1 Zer da FastAPI eta zergatik erabili?

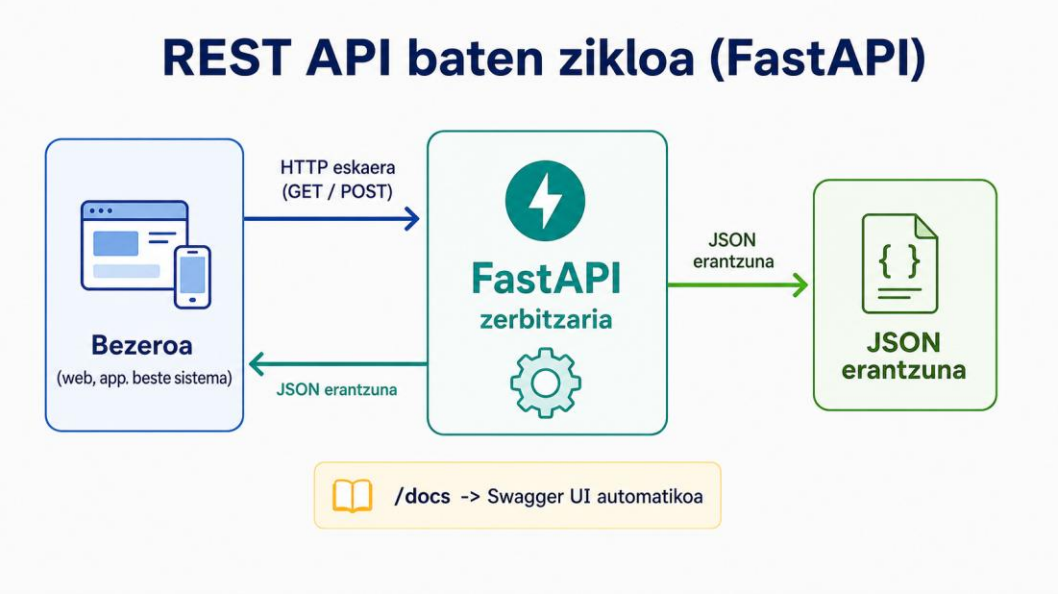
FastAPI Python-en REST API-ak eraikitzeko framework modernoa da. Flask-en sinpletasuna eta Django-ren osotasuna uztartzen ditu, eta lau abantaila nagusi ditu: **azkartasuna** (Node.js eta Go-ren pareko errendimendua), **mota-balioztatze automatikoa** (Python-en type hints erabiliz), **dokumentazio automatikoa** (`/docs` ibilbidean Swagger UI sortzen du berehala) eta **euskarri asinkronoa** (`async/await`).

```
pip install fastapi "uvicorn[standard]" pydantic

#hautazkoa: pip install email-validator
```

Aplikazioa abiarazteko `uvicorn` komandoa erabiltzen dugu. `--reload` aukerak zerbitzaria automatikoki berrabiarazten du kodea aldatzean (garapenerako; ekoizpenean desaktibatu):

```
uvicorn aplikazioa:app --reload --port 8000
```



12. Irudia: REST API baten oinarrizko zikloa — bezeroaren HTTP eskaera, FastAPI-ren prozesamendua, eta JSON erantzuna, Swagger UI dokumentazioarekin.

> 💡 ***Ideia nagusia*** *FastAPI-ren bereizgarri nagusia type hints-en gaineko balioztatze automatikoa da: zuk Python-en motak idazten dituzu, eta FastAPI-k sarrera balioztatu, dokumentatu eta errore argiak itzultzen ditu, kode gehigarririk gabe.*
>
> ☐ ***Ariketa 3.1 · 10 min · banaka***
>
> ***Eginkizuna****: Instalatu FastAPI eta uvicorn, sortu* `/` *ibilbide bat* `{"mezua": "Kaixo"}` *itzultzen duena, eta abiarazi* `uvicorn`*-ekin. Bisitatu* `http://localhost:8000/docs` *eta esploratu Swagger UI.*
>
> ***Eztabaidatzeko****: Zer abantaila du dokumentazioa automatikoki sortzeak eskuz idaztearen aldean?*

3.2 Oinarrizko ibilbideak eta HTTP metodoak

API batek eskaintzen dituen eragiketak HTTP metodoen bidez antolatzen dira: **GET** (irakurri), **POST** (sortu), **PUT** (eguneratu) eta **DELETE** (ezabatu). FastAPI-n metodo bakoitzerako dekoratzaile bat erabiltzen da (`@app.get`, `@app.post`...), eta funtzioek itzultzen dituzten datuak automatikoki JSON bihurtzen dira. Ondoko adibideak produktu-katalogo baten gaineko CRUD eragiketak erakusten ditu.

```
# aplikazioa.py
from fastapi import FastAPI, HTTPException, Query, Path
from typing import Optional

app = FastAPI(title="Nire Lehen API")

produktuak = {
    1: {"izena": "Ordenagailu eramangarria", "prezioa": 899.99, "stock": 15},
    2: {"izena": "Teklatua", "prezioa": 49.99, "stock": 50},
}

@app.get("/")
def hasiera():
    return {"mezua": "Ongi etorri APIra!"}

@app.get("/produktuak")
def produktu_guztiak(prezio_max: Optional[float] = Query(None)):
    emaitzak = list(produktuak.values())
    if prezio_max is not None:
        emaitzak = [p for p in emaitzak if p["prezioa"] <= prezio_max]
    return {"kopurua": len(emaitzak), "produktuak": emaitzak}

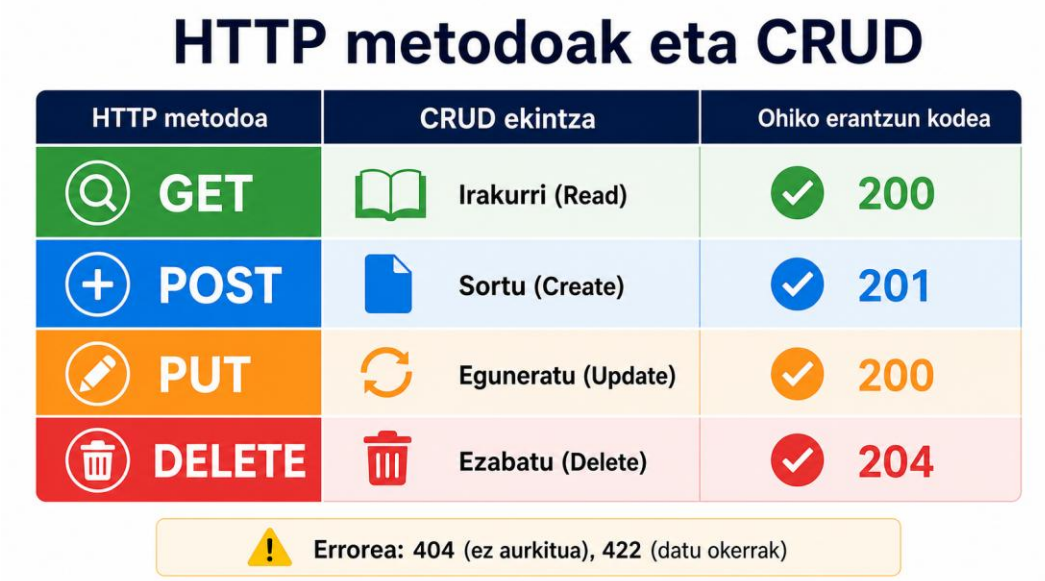
@app.get("/produktuak/{produktu_id}")
def produktu_bat(produktu_id: int = Path(..., ge=1)):
    if produktu_id not in produktuak:
        raise HTTPException(status_code=404, detail="Produktua ez da aurkitu")
    return produktuak[produktu_id]

@app.post("/produktuak", status_code=201)
```

```python
def produktu_sortu(produktu: dict):
    id_berria = max(produktuak.keys()) + 1
    produktuak[id_berria] = produktu
    return {"id": id_berria, **produktu}

@app.delete("/produktuak/{produktu_id}", status_code=204)
def produktu_ezabatu(produktu_id: int):
    if produktu_id not in produktuak:
        raise HTTPException(status_code=404, detail="Produktua ez da aurkitu")
    del produktuak[produktu_id]
```

`Query` eta `Path` objektuek parametroei murrizketak eta dokumentazioa gehitzen dizkiete. `HTTPException`-ek errore-egoeretan erantzun-kode egokia (404, 422...) itzultzen du, eta `status_code`-k arrakasta-kodea zehazten du (201 sortzean, 204 ezabatzean).



13. Irudia: Lau HTTP metodo nagusiak (GET, POST, PUT, DELETE) eta CRUD eragiketekin duten lotura, egoera-kode tipikoekin (200, 201, 204, 404).

🔍 ***Adibidea: dendaren stock-zerbitzua*** *Online-denda batek stock-a beste sistemekin partekatu behar du. `GET /produktuak` deituz, beste edozein sistemak (web-orria, app mugikorra, hornitzailearen sistema) JSON formatuan jasotzen du katalogoa. Produktu berri bat gehitzeko `POST` eta ezabatzeko `DELETE`. API berbera erabiltzen dute bezeroaren web-orriek eta barne-sistemek: hori da API-ren boterea.*

💡 ***Ideia nagusia*** *CRUD eragiketak (Create-Read-Update-Delete) HTTP metodoetara mapatzen dira (POST-GET-PUT-DELETE). Egoera-kodeak (200, 201, 404...) ondo erabiltzea API profesional baten ezaugarria da.*

☐ ***Ariketa 3.2 · 20 min · taldeka*** ***Eginkizuna**: Eraiki liburutegi baten API bat liburuekin (id, izenburua, egilea, urtea). Inplementatu GET (guztiak eta banaka), POST eta*

DELETE. Probatu Swagger UI-tik. ***Eztabaidatzeko****: Zergatik itzultzen da 204 ezabatzean eta 201 sortzean? Zer adierazten du kode bakoitzak?*

3.3 Pydantic: datu-balioztatze automatikoa

Pydantic-ek sarrerako datuak automatikoki balioztatzen ditu eta mota okerrak daudenean errore argiak itzultzen ditu. Hau funtsezkoa da edozein ekoizpen-API-tan: "erabiltzaileek bidalitako datuez ez fidatzea" segurtasun-printzipio nagusietako bat da. Pydantic-ek lan hori modu deklaratiboan egitea ahalbidetzen du, motei lotuta.

```
from fastapi import FastAPI
from pydantic import BaseModel, Field, field_validator

app = FastAPI()

class ProduktuSarrera(BaseModel):
    izena:   str   = Field(..., min_length=2, max_length=100)
    prezioa: float = Field(..., gt=0, description="Prezioa eurotan") #gt: greater than
    stock:   int   = Field(default=0, ge=0) #ge: greater or equal than
    kategoria: str | None = None #hutsa izan leike (string edo hutsa onartzen ditu)

    @field_validator("izena")
    @classmethod
    def izena_garbitu(cls, balioa: str) -> str:
        return balioa.strip().title()

@app.post("/produktuak", status_code=201)
def produktu_sortu(produktua: ProduktuSarrera):
    # Hona iristen bada, datuak DAGOENEKO balioztatuta daude
    return {"izena": produktua.izena, "prezioa": produktua.prezioa}
```

Datu-ereduak bereiztea (sarrera, erantzuna, errorea) praktika ona da: sarrera eta irteera askotan ez datoz bat (adibidez `id` eta `sortze_data` zerbitzariak sortzen ditu). `@field_validator` dekoratzaileak eraldaketak edo egiaztapen gehigarriak aplikatzen ditu. Ordea `Field` erabiltzen da baldintza oso xumeentzako, bestera jo aurretik egin ohi direnak.

🔍 ***Adibidea: prezio negatiboa saihestu*** *Erabiltzaile batek akatsez `-50` prezioa bidaltzen du. Pydantic gabe, prezio negatiboa datu-basera helduko litzateke eta arazoak sortuko lituzke. `Field(..., gt=0)` murrizketarekin, FastAPI-k automatikoki 422 errorea itzultzen du mezu argi batekin: "prezioa 0 baino handiagoa izan behar da". Garatzaileak ez du lerro bakar bat ere idatzi behar egiaztapen horretarako.*

💡 ***Ideia nagusia*** *Pydantic-ek sarrerak deklaratiboki balioztatzen ditu: automatikoki json-etik bihurtu, motak eta murrizketak adierazi, eta FastAPI-k automatikoki balioztatu eta errore argiak itzuli. Inoiz ez fidatu erabiltzailearen sarreraz.*

☐ ***Ariketa 3.3 · 20 min · banaka***

Eginkizuna: *Sortu Pydantic eredu bat erabiltzaile baterako: izena (gutxienez 3 karaktere), adina (0-120 artean), email (string). Gehitu balioztatzaile bat emailak `@` daukala egiaztatzeko. Probatu balio okerrekin.*

Eztabaidatzeko: *Zergatik da arriskutsua erabiltzailearen datuez fidatzea? Zer gerta liteke balioztatze gabe?*

3.4 ML eredua API bidez eskaini

Hau gaiaren funtsetako bat da: aurreko ataletan entrenatutako ML eredu bat REST API baten bidez eskaintzea. Hau da MLOps prozesuaren urrats erabakigarria, ereduek balio erreala ekoizpenean hedatuta soilik baitute. Ondoko adibidean, Scikit-Learn pipeline bat (`joblib`-ekin serializatua) FastAPI-k abiaraztean kargatzen du, eta iragarpen-eskaerak prozesatzeko ibilbideak eskaintzen ditu.

🔍 ***Agente-erabilera ona*** *(gogoratu 1. gaiko §2.5 ohiturak) FastAPI endpoint baten scaffold-a eskatzea kasu paradigmatikoa da agente-erabilera onerako. Eman agenteari kontestu zehatza: zer eredu kargatuko duzu (joblib path), zer sarrera espero duzu (Pydantic eredua emanik: zutabe-zerrenda + tipoak), zer formatu itzuli (JSON estruktura), eta zer errore-kudeaketa nahi duzun. Ez eskatu "egin FastAPI app bat" — eman pieza guztiak (1. gaiko §2.5.1). Behin sortuta,* ***beti exekutatu lokalean eta probatu `curl`-ekin endpoint guztiak*** *(§2.5.5 — egiaztapena). Hardcoded API gakorik agertzen ote den kontuz begiratu (§2.5.6).*

```
# ml_api.py
from contextlib import asynccontextmanager
from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel, Field
import joblib
import numpy as np
import os

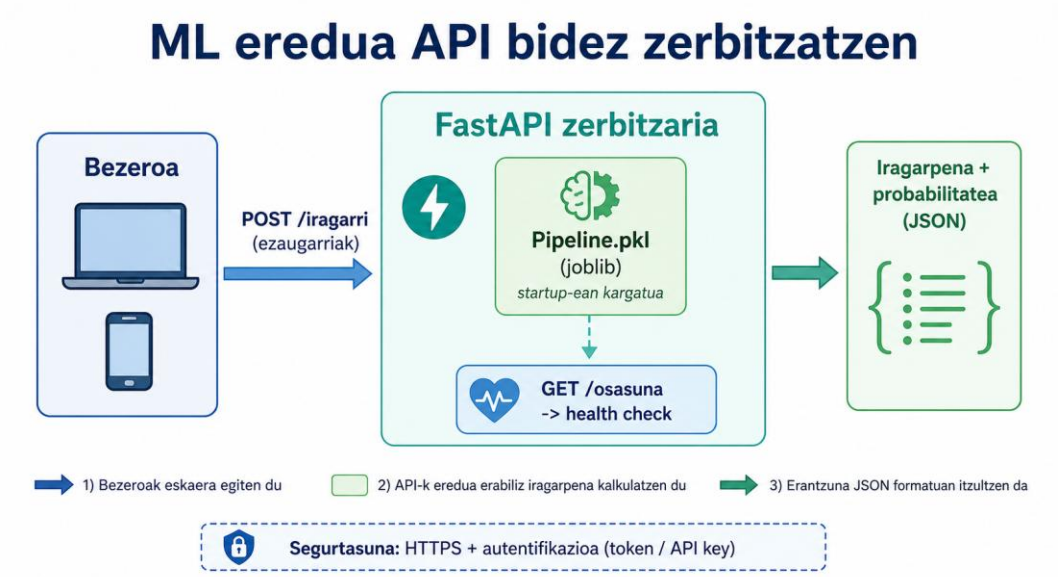
EREDU_BIDE = "models/pipeline.pkl"
eredua = None

# Startup eta Shutdown kudeatzeko lifespan kudeatzailea
@asynccontextmanager
async def lifespan(app: FastAPI):
    # --- STARTUP LOGIKA ---
    global eredua
    if os.path.exists(EREDU_BIDE):
        eredua = joblib.load(EREDU_BIDE)
```

```
        print(f"Eredua kargatuta: {EREDU_BIDE}")
    else:
        print(f"Abisua: Fitxategia ez da aurkitu hemen: {EREDU_BIDE}")
    yield  # Aplikazioa martxan dago atal honetan
    # --- SHUTDOWN LOGIKA ---
    eredua = None
    print("Zerbitzaria itzaltzen: baliabideak garbituta.")

app = FastAPI(title="ML Iragarpen API", version="1.0.0", lifespan=lifespan)

class IragarpenEskaera(BaseModel):
    ezaugarriak: list[float] = Field(..., min_length=30, max_length=30)
class IragarpenErantzuna(BaseModel):
    iragarpena: int
    etiketa: str
    probabilitatea: float
@app.get("/osasuna")
def osasun_egiaztatu():
    if eredua is None:
        raise HTTPException(
            status_code=503,
            detail="Eredua ez dago kargatuta.",
        )
    return {"egoera": "martxan", "eredu_prest": True}
@app.post("/iragarri", response_model=IragarpenErantzuna)
def iragarri(eskaera: IragarpenEskaera):
    if eredua is None:
        raise HTTPException(
            status_code=status.HTTP_503_SERVICE_UNAVAILABLE,
            detail="Eredua ez dago prest.",
        )
    X = np.array(eskaera.ezaugarriak).reshape(1, -1)
    irag = int(eredua.predict(X)[0])
    proba = float(eredua.predict_proba(X)[0][1])
    return IragarpenErantzuna(
        iragarpena=irag,
        etiketa="Txarra" if irag == 1 else "Ona",
        probabilitatea=round(proba, 4),
    )
```



14. Irudia: ML eredu baten zerbitzaratze-arkitektura — `joblib` pipeline-a abiaraztean kargatu, `/iragarri` ibilbidea eta `/osasuna` health-check-a, bezeroaren eskaeratik iragarpenera.

Alderdi azpimarragarriak: eredua `startup`-ean kargatzen da (ez eskaera bakoitzean, errendimenduagatik); `/osasuna` ibilbidea (health check) funtsezkoa da ekoizpenean, Kubernetes edo Docker bezalako orkestratzaileek zerbitzuaren egoera egiaztatzeko; eta eredua prest ez badago, 503 kodea itzultzen da (zerbitzu erabilgaitasun-eza).

🔍 ***Adibidea: minbizi-eredutik web-zerbitzura*** *1.3 atalean entrenatu zenuen bular-minbizi sailkatzailea pipeline batean bilduta dago. Orain ospitaleko beste sistemek erabili nahi dute. `joblib.dump`-ekin gorde, FastAPI-k abiaraztean kargatu, eta `POST /iragarri` deituz, edozein sistemak (web-orria, EHR sistema) iragarpen bat lortzen du JSON formatuan. Ereduak laborategitik ospitaleko ekoizpenera salto egin du.*

💡 ***Ideia nagusia*** *Eredua `startup`-ean behin kargatu (ez eskaera bakoitzean). `/osasuna` health-check-a ezinbestekoa da orkestratzaileentzat. 503 kodea zerbitzua oraindik prest ez dagoela adierazteko.*

⚠ ***Kontuz: pipeline osoa gorde, ez eredua bakarrik*** *ML API batean ohiko akatsa: eredua bakarrik gordetzea, aurreprozesatzailea ahaztuz. Orduan API-ak datu gordinak jasotzen ditu, baina ereduak eskalatutakoak espero ditu. Beti gorde **pipeline osoa** (aurreprozesatzailea + eredua) `joblib`-ekin, 1.5 atalean ikusi bezala.*

☐ ***Ariketa 3.4 · 25 min · taldeka***

***Egoera**: 1.5 ariketan gorde zenuten pipeline-a duzue.*

***Eginkizuna**: Eraiki FastAPI API bat pipeline hori `startup`-ean kargatzen duena eta `POST /iragarri` ibilbide bat eskaintzen duena. Gehitu `/osasuna` health-check-a. Probatu `curl` edo Swagger UI-tik.*

***Eztabaidatzeko**: Zergatik kargatu eredua `startup`-ean eta ez eskaera bakoitzean? Zer gertatuko litzateke milaka eskaerarekin?*

3.5 Autentifikazioa eta middleware-a

API bat publikoki eskaintzen denean, segurtasuna funtsezkoa da. Atal honetan hiru kontzeptu lantzen ditugu: **CORS** politika (zein nabigatzailek dei dezakeen API-ari), **middleware**-ak (eskaera guztietan exekutatzen den logika orokorra, hala nola erregistroa edo denbora-neurketa), eta **Bearer Token** bidezko autentifikazioa (ibilbide babestuetarako sarbidea kontrolatzeko).

```
from fastapi import FastAPI, Depends, HTTPException, status
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
from fastapi.middleware.cors import CORSMiddleware
import time, os

from dotenv import load_dotenv

load_dotenv()

app = FastAPI()

# CORS middleware-a (web nabigatzaileetarako)
app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://nire-aplikazioa.com"],
    allow_methods=["*"], allow_headers=["*"],
)

# Denbora-neurketako middleware-a
@app.middleware("http")
async def denbora_neurtu(eskaera, deitu_hurrengoa):
    hasiera = time.time()
    erantzuna = await deitu_hurrengoa(eskaera)
    erantzuna.headers["X-Prozesatze-Denbora"] = str(round(time.time() - hasiera, 4))
    return erantzuna

# Token bidezko autentifikazioa
TOKEN_BALIOA = os.getenv("API_TOKEN", "")    # ingurune-aldagaitik!
segurtasuna = HTTPBearer()

def token_egiaztatu(kredentzialak: HTTPAuthorizationCredentials = Depends(segurtasuna)):
    if kredentzialak.credentials != TOKEN_BALIOA:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Token
baliogabea")
    return kredentzialak.credentials

@app.get("/babestua", dependencies=[Depends(token_egiaztatu)])
```

```
def ibilbide_babestua():
    return {"mezua": "Token egiazkoa! Ongi etorri."}
```

Ekoizpenean **inoiz ez** dira tokenak kodean idatzi behar — beti ingurune-aldagaietatik (`os.environ`) edo secret manager-etatik (Vault, AWS Secrets Manager) irakurri. Sistema profesionaletan JWT edo OAuth2 erabili ohi dira, baina Bearer Token sinpleak nahikoa dira barne-zerbitzuetarako.

🔍 ***Adibidea: API-gakoa filtratuta*** *Garatzaile batek tokena kodean idatzi eta GitHub-era igo zuen. Ordu gutxitan, bot-ek detektatu eta erasotzaileek API-az aprobetxatu ziren, fakturak igoz. Ikasgaia: tokenak beti ingurune-aldagaietan (`os.getenv`), inoiz ez kodean. Igo bada,* ***berehala*** *aldatu giltza.*

❗ ***Ideia nagusia*** *Sekretuak (tokenak, API-gakoak) beti ingurune-aldagaietatik kargatu, inoiz ez idatzi kodean. Middleware-ak eskaera guztietan exekutatzen den logika orokorrarentzat dira (log-ak, denbora, CORS).*

☐ ***Ariketa 3.5 · 20 min · banaka***

Eginkizuna*: Babestu aurreko ariketako `/iragarri` ibilbidea Bearer Token batekin. Token-balioa `os.getenv`-tik kargatu. Probatu tokenarekin eta gabe (401 jaso behar duzu token gabe).*

Eztabaidatzeko*: Zergatik ez da segurua tokena kodean idaztea, GitHub publikoa ez denean ere?*

Atalaren laburpena: FastAPI-rekin REST API profesionalak eraikitzeko oinarriak ikusi ditugu: ibilbideak eta HTTP metodoak, Pydantic balioztatzea, ML eredu baten zerbitzaratzea, eta autentifikazio nahiz middleware mekanismoak. Oinarri hauekin edozein ML soluzio web-zerbitzu bihur daiteke.

4. Streamlit: Datu-aplikazioak Azkar Sortu

Atal honetan Streamlit liburutegia landuko dugu, datu-zientzialariek prototipoak eta barne-tresnak azkar sortzeko erabiltzen dutena. FastAPI-k backend-a eskaintzen badu ere, askotan ezagutza teknikoa ez duen jendeari emaitzak modu grafiko eta interaktiboan aurkeztu behar zaizkie; horretarako Streamlit aukera ezin hobea da, web-ezagutzarik gabe Python soilik erabiliz aplikazio profesionalak sortzeko.

4.1 Streamlit zer da eta oinarrizko egitura

Streamlit datu-aplikazioak eta prototipoak azkar sortzeko Python liburutegia da. Kode-lerro gutxirekin web-aplikazio interaktiboak eraiki daitezke, web-garapenik jakin gabe. Bere filosofia goitik beherako exekuzio-eredua da: script-a goitik behera berriz exekutatzen da erabiltzaileak edozein osagairekin interakzionatzen duen bakoitzean, eta horrek programazio-eredua oso intuitiboa egiten du.

```
pip install streamlit


streamlit run aplikazioa.py   # Automatikoki irekitzen du http://localhost:8501
```

`st.set_page_config()` da scriptean Streamlit liburutegia erabiltzen hasteko lehen metodo-deia:

```
# aplikazioa.py
import streamlit as st
import pandas as pd


# Beti aurreneko deia hau:
st.set_page_config(page_title="Nire Aplikazioa", page_icon="□", layout="wide")

st.title("Nire Lehen Streamlit Aplikazioa")
st.markdown("**Lodia**, *etzana* eta `kode` formatua Markdown bidez.")
st.info("□ Informazio-mezua")
st.success("✓ Arrakasta-mezua")

df = pd.DataFrame({"izena": ["Ane", "Mikel"], "soldata": [28000, 35000]})
st.dataframe(df, use_container_width=True)
st.metric("Batezbesteko soldata", f"{df['soldata'].mean():,.0f} €", delta="2.3%")

col1, col2, col3 = st.columns(3)
col1.metric("Langileak", 2)
col2.metric("Soldata max", "35.000 €")
col3.metric("Soldata min", "28.000 €")
```



15. Irudia: Streamlit-en exekuzio-eredua — erabiltzaileak osagai batekin interakzionatzean script osoa goitik behera berriz exekutatzen da, eta interfazea eguneratzen.

☐ ***Adibidea: zuzendaritzari emaitzak erakutsi*** *ML eredu bat entrenatu duzu, baina zuzendaritzak ez daki Python-ik. Streamlit-ekin, ordu batean web-aplikazio bat sortzen duzu: KPI-ak `st.metric`-ekin, taula `st.dataframe`-ekin, eta dena nabigatzailetik ikusgai. Zuzendariak klik batekin ikusten ditu emaitzak, kodea ikusi gabe. Hori da Streamlit-en boterea: teknikoa eta ez-teknikoa konektatzen ditu.*

💡 ***Ideia nagusia*** *Streamlit-ek Python datu-egitura natiboak zuzenean ulertzen ditu: `st.dataframe(df)` aski da DataFrame bat interaktiboki erakusteko. Goitik beherako exekuzio-ereduak programazioa intuitiboa egiten du.*

☐ ***Ariketa 4.1 · 15 min · banaka*** ***Eginkizuna****: Sortu Streamlit aplikazio bat izenburuarekin, DataFrame batekin (5 langile fikziozko) eta hiru `st.metric`-ekin (kopurua, batezbesteko adina, soldata maximoa). Abiarazi eta ikusi nabigatzailean.*
Eztabaidatzeko*: Zertarako da erabilgarria `st.metric`-en `delta` parametroa? Zein kasutan erabiliko zenuke?*

4.2 Sarrera-osagaiak eta egoera

Erabiltzailearekin elkarrekintza Streamlit aplikazio gehienen oinarria da. Liburutegiak osagai-aukera zabala eskaintzen du (testu-eremuak, slider-ak, hautagailuak, botoiak), eta horien balioa zuzenean Python-eko aldagaietan jasotzen da. Hala ere, script-a osorik berriz exekutatzen denez interakzio bakoitzean, datuak elkarrizketen artean gordetzeko mekanismo bat behar da: hor sartzen da `st.session_state` hiztegia.

```
import streamlit as st

st.title("Sarrera-osagaiak")

with st.sidebar:
    st.header("Konfigurazioa")
    hizkuntza = st.selectbox("Hizkuntza", ["Euskara", "Gaztelania", "Ingelesa"])

izena = st.text_input("Zure izena:", placeholder="Ane Etxebarria")
adina = st.number_input("Adina:", min_value=0, max_value=120, value=25)
soldata = st.slider("Urteko soldata (€):", 15_000, 100_000, 30_000, step=1_000)
departamentua = st.selectbox("Departamentua:", ["IT", "HR", "Finantza"])
baldintza = st.checkbox("Baldintzak onartzen ditut")

if st.button("Bidali", type="primary", disabled=not baldintza):
    st.success(f"Eskerrik asko, {izena}!")
    st.json({"izena": izena, "adina": adina, "soldata": soldata})

# Session state: aldagaiak saioaren artean gorde
if "klik_kopurua" not in st.session_state:
    st.session_state.klik_kopurua = 0
if st.button("Klik egin!"):
    st.session_state.klik_kopurua += 1
st.write(f"Klik kopurua: {st.session_state.klik_kopurua}")
```

☐ ***Adibidea: kontagailua mantentzen*** *Streamlit-en script-a interakzio bakoitzean berriz exekutatzen denez, aldagai arrunt bat 0-ra itzuliko litzateke klik bakoitzean.* `st.session_state.klik_kopurua` *erabiliz, ordea, balioa saioan zehar mantentzen da. Berdina gertatzen da formulario-historia, erabiltzailearen aukerak edo elkarrizketa-historia gordetzeko: dena* `session_state`*-n.*

💡 ***Ideia nagusia*** *Script-a interakzio bakoitzean berriz exekutatzen denez, aldagaiak galtzen dira.* `st.session_state` *saioan zehar irauten duen egoera gordetzeko da (kontagailuak, historiak, aukerak).*

⚠ ***Kontuz:*** `set_page_config` ***lehenengo*** `st.set_page_config()` *script-aren lehen Streamlit-deia izan behar da derrigorrez. Beste edozein* `st.` *deiaren ondoren jartzen baduzu, errorea jaurtiko du. Beti goian.*

☐ ***Ariketa 4.2 · 20 min · taldeka*** ***Eginkizuna****: Sortu formulario bat sidebar-ekin (hizkuntza, gaia) eta orri nagusian (izena, adina, interesak* `multiselect`*-ekin). Botoia sakatzean, erakutsi datuak* `st.json`*-ekin. Gehitu klik-kontagailu bat* `session_state`*-rekin.* ***Eztabaidatzeko****: Zergatik behar da* `session_state`*? Zer gertatuko litzateke aldagai arrunt bat erabiliko bagenu kontagailurako?*

4.3 Grafikoak eta datuak erakutsi

Datuen bistaratzea datu-aplikazio gehienen bihotza da. Streamlit-ek hainbat aukera onartzen ditu: bere barne-grafikoak (azkar baina mugatuak), Matplotlib/Seaborn (estatikoak, pertsonalizagarriak) eta Plotly (interaktiboak, ekoizpenerako gomendagarrienak). Aukera bakoitzak bere kasua du: barne-grafikoak prototipo azkarrentzat, Matplotlib txosten estatikoetarako, eta Plotly amaierako produktuetarako.

```
import streamlit as st
import pandas as pd, numpy as np
import matplotlib.pyplot as plt
import plotly.express as px

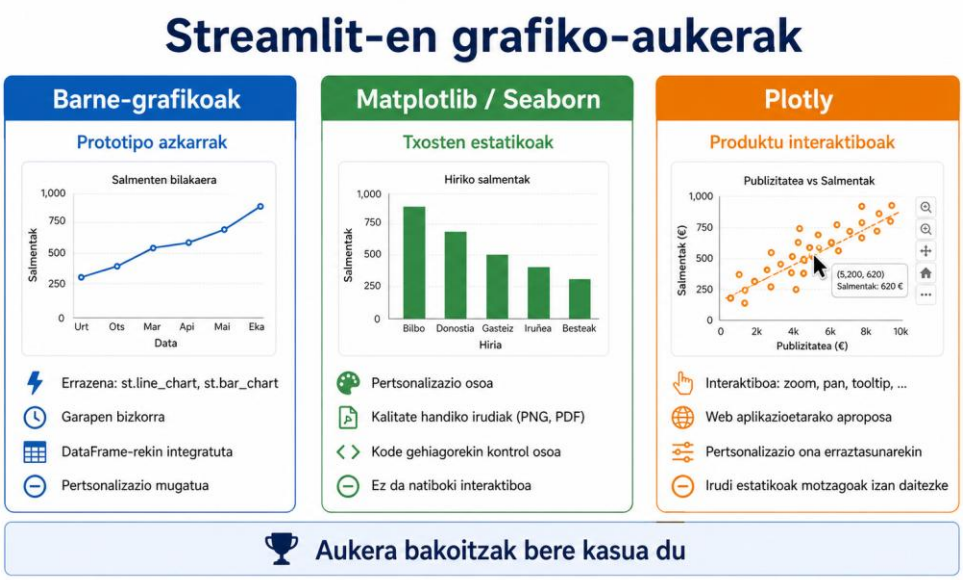
st.title("Grafikoak Streamlit-en")

np.random.seed(42)
df = pd.DataFrame({
    "data":      pd.date_range("2024-01-01", periods=90),
    "salmenta":  np.random.randint(100, 500, 90),
    "produktua": np.random.choice(["A", "B", "C"], 90),
})

# Streamlit barne-grafikoak (azkarrak)
st.line_chart(df.set_index("data")["salmenta"])
st.bar_chart(df.groupby("produktua")["salmenta"].sum())

# Matplotlib (pertsonalizagarria) — gogoratu plt.close!
fig, ax = plt.subplots(figsize=(8, 4))
ax.hist(df["salmenta"], bins=20, color="#2ecc71", edgecolor="white")
ax.set_title("Salmenten banaketa")
st.pyplot(fig); plt.close(fig)

# Plotly (interaktiboa: zoom, hover)
fig_plotly = px.scatter(df, x="data", y="salmenta", color="produktua",
                        size="salmenta", title="Salmenta denboran zehar")
st.plotly_chart(fig_plotly, use_container_width=True)
```



16. Irudia: Streamlit-en hiru grafiko-aukerak (barne-grafikoak, Matplotlib, Plotly) konparatuta — azkartasuna, pertsonalizazioa eta interaktibitatea, eta noiz erabili bakoitza.

☐ ***Adibidea: salmenta-panela elkarreragingarria*** *Salmenta-taldeak panel bat nahi du salmentak denboran zehar ikusteko. Plotly-rekin, grafikoa interaktiboa da: zooma egin, produktu baten gainetik pasatzean datu zehatzak ikusi, eta PNG gisa esportatu. Streamlit barne-grafikoekin azkar prototipatuko zenuke; Plotly-rekin produktu profesionalagoa emango zenioke.*

💡 ***Ideia nagusia*** *Streamlit barne-grafikoak prototipoetarako; Matplotlib txosten estatikoetarako; Plotly amaierako produktu interaktiboetarako; Kibana integrazio osoko zerbitzari profesionalerako. Matplotlib erabiltzean beti plt.close(fig) deitu memoria-jarioak saihesteko.*

4.4 ML aplikazio osoa Streamlit-ekin

Atal honetan aurreko kontzeptuak konbinatuko ditugu ML aplikazio profesional batean: eredu baten karga (cachearekin optimizatua), iragarpen-interfazea, eredu-ebaluazioko orria eta dokumentazioa. `@st.cache_resource` dekoratzaileak ereduaren karga behin bakarrik egiten du saioan zehar, errendimendua nabarmen hobetuz.

```
# ml_aplikazioa.py
import streamlit as st
import pandas as pd, numpy as np
import matplotlib.pyplot as plt, seaborn as sns
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix

st.set_page_config(page_title="Minbizi Detekzioa", page_icon="□", layout="wide")

@st.cache_resource   # Eredua behin bakarrik kargatu/entrenatu
def eredua_kargatu():
    d = load_breast_cancer()
    X_ent, X_test, y_ent, y_test = train_test_split(d.data, d.target, test_size=0.2,
random_state=42)
    eskala = StandardScaler()
    X_ent = eskala.fit_transform(X_ent); X_test = eskala.transform(X_test)
    rf = RandomForestClassifier(n_estimators=100, random_state=42).fit(X_ent, y_ent)
    return rf, eskala, d, X_test, y_test

eredua, eskala, d, X_test, y_test = eredua_kargatu()

st.title("□ Bular-minbizi Detekzio Tresna")
orria = st.sidebar.radio("Orria:", ["□ Iragarri", "□ Ebaluazioa", "□ Ereduaz"])

if orria == "□ Iragarri":
    st.header("Iragarpen pertsonalizatua")
    with st.form("forma"):
        balioak = [st.number_input(izena, value=float(d.data[:, i].mean()), key=f"e{i}")
                   for i, izena in enumerate(d.feature_names[:5])]   # adibide soilik
        bidali = st.form_submit_button("Iragarri", type="primary")
    if bidali:
        st.success("Iragarpena egina (adibide-interfazea)")

elif orria == "□ Ebaluazioa":
    st.header("Ereduaren errendimendua")
    y_irag = eredua.predict(X_test)
    col1, col2 = st.columns(2)
    col1.metric("Zehaztasuna", f"{accuracy_score(y_test, y_irag):.3f}")
    col2.metric("F1", f"{f1_score(y_test, y_irag):.3f}")
    fig, ax = plt.subplots(figsize=(5, 4))
    sns.heatmap(confusion_matrix(y_test, y_irag), annot=True, fmt="d", cmap="Blues", ax=ax)
    st.pyplot(fig); plt.close(fig)

else:
    st.markdown("**Algoritmoa:** Random Forest · **Datuak:** Breast Cancer (569 lagin)")
    st.info("Aplikazio hau hezkuntza-helburuekin soilik sortua da.")
```



17. Irudia: ML aplikazio osoaren egitura Streamlit-en — sidebar-eko nabigazioa hiru orriekin (iragarpena, ebaluazioa, dokumentazioa) eta `@st.cache_resource` bidezko ereduaren karga.

☐ ***Adibidea: ospitalerako tresna*** *Ospitale batek minbizi-detekzio tresna nahi du klinikoek erabiltzeko. Streamlit-ekin: orri bat iragarpenak egiteko (ezaugarriak sartu, emaitza jaso probabilitatearekin), beste bat ereduaren errendimendua erakusteko (metrikak, nahasketa-matrizea), eta hirugarren bat ereduari buruzko informazioarekin. Dena nabigatzailetik, instalaziorik gabe.*

💡 ***Ideia nagusia*** `@st.cache_resource` *ereduentzat eta* `@st.cache_data` *datuentzat: behin bakarrik kargatzen dute, errendimendua hobetuz. Erabiltzaileari emaitzen zentzua eman (probabilitateak, metrikak), ez bakarrik klase iragarria.*

☐ ***Ariketa 4.4 · 30 min · taldeka*** ***Eginkizuna****: Eraiki ML aplikazio osoa sidebar-eko nabigazioarekin: orri bat iragarpenetarako, beste bat ebaluaziorako (metrikak + nahasketa-matrizea). Erabili* `@st.cache_resource` *ereduarentzat.* ***Eztabaidatzeko****: Noiz erabili Streamlit eta noiz FastAPI? Konbina al daitezke biak?*

Atalaren laburpena: Streamlit-ek datu-aplikazioak modu azkar eta argian sortzea ahalbidetzen du. Sarrera-osagaiak, bistaratzea, egoera eta cachea ezagututa, edozein ML emaitza interaktiboki aurkez daiteke. Barne-tresnetarako bikaina; ekoizpen-trafiko handian FastAPI backend bat ondoan izatea komeni da.

5. LangChain: LLM Aplikazioak Eraikitzen (Gemini-rekin)

Atal honetan LangChain-en oinarriak ikasiko ditugu, LLM-en gainean aplikazio konplexuak eraikitzeko *de facto* bihurtu den esparrua. GenAI aplikazio gehienek LLM dei sinpleak baino gehiago behar dute: prompt-en kudeaketa, hainbat pausoren kateaketa, elkarrizketa-historia, kanpoko tresnetara sarbidea. LangChain-ek osagai horiek modulu bateratuetan eskaintzen ditu. Ikastaro honetan **Gemini** erabiliko dugu LLM gisa (zentroaren lizentziagatik), `langchain-google-genai` integrazioaren bidez.

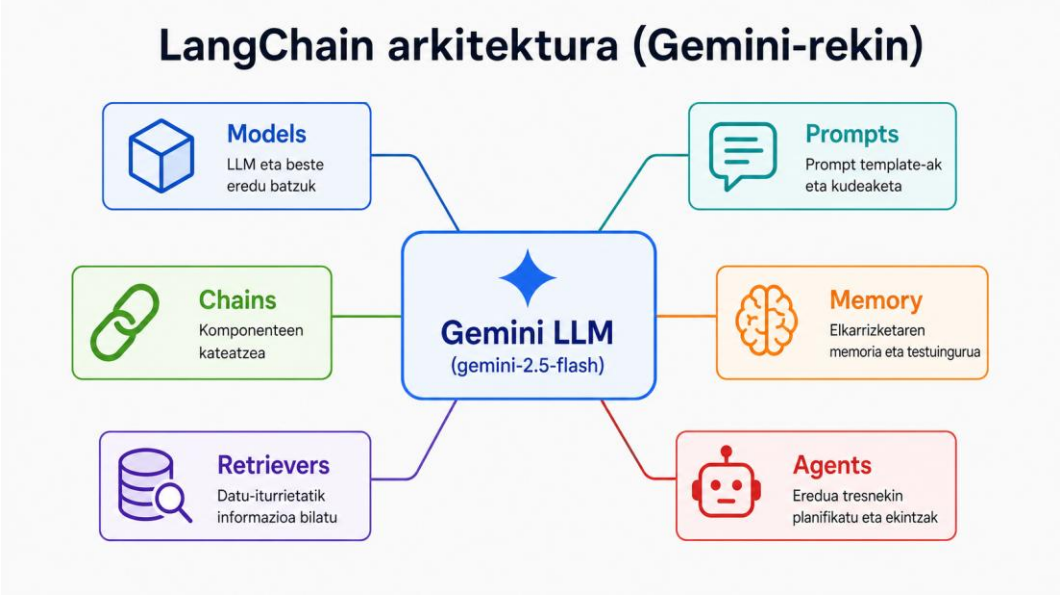
5.1 LangChain zer da eta arkitektura

LangChain LLM-en gaineko aplikazioak eraikitzeko Python esparrua da. Honako osagaiak biltzen ditu:

Osagaia	Zeregina
Models	LLM eta chat-ereduei konexioa (Gemini, OpenAI, lokalekoak...)
Prompts	Galdera-txantiloiak dinamikoki sortu
Chains	Hainbat pauso segidan kateatu
Memory	Elkarrizketa-historia gorde
Retrievers	Kanpoko dokumentuetatik informazioa berreskuratu
Agents	LLM-ak erabakiak hartu eta tresnak erabili

Beharrezko paketeak: `langchain` esparru nagusia, `langchain-google-genai` Gemini ereduekin lan egiteko integrazioa, `langchain-community` hirugarrenen integrazioak, eta `python-dotenv` API-gakoak `.env` fitxategitik seguru kargatzeko. Gemini-ren API-gakoa [Google AI Studio](https://aistudio.google.com/apikey)-tik lortzen da, doan.

```
pip install langchain langchain-google-genai langchain-community python-dotenv
```



18. Irudia: LangChain-en sei osagai nagusiak (Models, Prompts, Chains, Memory, Retrievers, Agents) eta Gemini LLM-arekin nola konektatzen diren.

> 💡 ***Ideia nagusia*** *LangChain-ek LLM aplikazioen pieza errepikariak (prompt-ak, kateak, memoria, tresnak) modulu bateratuetan biltzen ditu. Ikastaro honetan Gemini erabiltzen dugu LLM gisa,* `langchain-google-genai` *integrazioarekin.*
>
> ☐ ***Ariketa 5.1 · 10 min · banaka***
>
> ***Eginkizuna****: Lortu Gemini-ren API-gako doako bat Google AI Studio-tik. Gorde* `.env` *fitxategi batean* `GOOGLE_API_KEY=...` *gisa. Egiaztatu* `.env` `.gitignore`*-n dagoela.*
>
> ***Eztabaidatzeko****: Zergatik ez da gakoa kodean idatzi behar? Nola kudeatzen dira gakoak ekoizpenean?*

5.2 Prompt template-ak eta chain-ak

LLM aplikazioetan, prompt-ak (LLM-ari bidaltzen zaizkion mezuak) askotan dinamikoki sortu behar dira erabiltzailearen sarreraren arabera. LangChain-ek `ChatPromptTemplate` eskaintzen du txantiloi parametrizagarriak idazteko, eta LCEL (LangChain Expression Language) sintaxiarekin (`|` operadorea) hainbat osagai kateatzeko aukera ematen du, modu deklaratiboan.

```
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain.prompts import ChatPromptTemplate
from langchain_core.messages import HumanMessage, SystemMessage
from langchain_core.output_parsers import StrOutputParser
from dotenv import load_dotenv

load_dotenv()   # GOOGLE_API_KEY .env fitxategitik irakurri
```

```
llm = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0.7)

# ---- Mezua zuzenean bidali ----
erantzuna = llm.invoke([
    SystemMessage(content="Laguntzaile tekniko bat zara. Erantzun laburki eta argiro."),
    HumanMessage(content="Zer da Python-eko list comprehension bat?")
])
print(erantzuna.content)

# ---- Prompt template parametrizatua ----
txantiloia = ChatPromptTemplate.from_messages([
    ("system", "Aditu bat zara {arloa} arloan. Erantzun {hizkuntza} hizkuntzan."),
    ("human",  "{galdera}")
])

# ---- LCEL: chain-a | operadorearekin ----
chain = txantiloia | llm | StrOutputParser()

emaitza = chain.invoke({
    "arloa":     "ikasketa automatikoa",
    "hizkuntza": "euskara",
    "galdera":   "Zer da Random Forest algoritmoa?"
})
print(emaitza)   # Testu soila, ez objektua
```

`temperature` parametroak erantzunen ausazkotasuna kontrolatzen du: 0 zehatzena eta deterministena (klasifikazio/erauzketarako), 1 sortzaileena (testu kreatiboetarako). `StrOutputParser`-ek chain-aren irteera testu hutsa izatea bermatzen du. LCEL chain-ek `invoke()`, `stream()` eta `batch()` metodoak uniformeki eskaintzen dituzte.



19. Irudia: LangChain LCEL chain baten fluxua — prompt template, Gemini LLM eta output parser piezak `|` operadorearekin kateatuta, sarreratik testu-erantzunera.

🔍 ***Adibidea: produktu-deskribapen automatikoak*** *Online-denda batek 500 produktu ditu eta deskribapen erakargarriak behar ditu. Chain bat sortzen duzu: txantiloi batean produktuaren ezaugarriak sartu, Gemini-ri pasa, eta deskribapen komertziala jaso euskaraz.* `chain.batch([...])` *erabiliz, 500 produktu denak batera prozesatzen dituzu. Eskuz astebeteko lana, kodez ordu batean.*

💡 ***Ideia nagusia***

LCEL-ek (`|` operadorea) prompt + LLM + parser piezak modu deklaratiboan kateatzen ditu. `temperature` *baxua (0) zehaztasunerako, altua (0.7-1) sormena behar denerako.*

⚠ ***Kontuz: API-deiek kostua dute*** *LLM-deietan, token bakoitzak kostua du (eta latentzia). 500 produktu prozesatzean, milaka token kontsumitzen dira. Beti egiaztatu zenbat kostatzen den probatu aurretik, eta erabili* `temperature=0` *deterministena behar denean (gainera, cachea errazten du).*

☐ ***Ariketa 5.2 · 25 min · taldeka***

Eginkizuna*: Sortu chain bat Gemini-rekin produktu baten ezaugarriak (izena, kategoria, prezioa) hartuta deskribapen komertzial bat euskaraz sortzen duena. Probatu 3 produktu desberdinekin* `batch`*-ekin.*

Eztabaidatzeko*: Zer* `temperature` *erabiliko zenukete? Zergatik desberdina deskribapen kreatiboetarako eta datu-erauzketarako?*

5.3 Memoria eta elkarrizketa-historia

Berez LLM-ak ez du oroimenik elkarrizketen artean: dei bakoitza independentea da. Chatbot bat eraikitzeko, mezuen historia gorde eta dei berri bakoitzean bidali behar dugu. Adibide modernoetan, `RunnableWithMessageHistory` erabiltzen da hori modu deklaratiboan integratzeko, `session_id` baten arabera erabiltzaile bakoitzaren historia bereiziz.

```
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.runnables.history import RunnableWithMessageHistory
from langchain_community.chat_message_histories import ChatMessageHistory

llm = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0.5)

txantiloia = ChatPromptTemplate.from_messages([
    ("system", "Laguntzaile adimendun bat zara. Erantzun labur eta argian."),
    MessagesPlaceholder(variable_name="history"),
    ("human",  "{sarrera}")
])

saio_historioak: dict[str, ChatMessageHistory] = {}
```

```
def historia_lortu(saio_id: str) -> ChatMessageHistory:
    if saio_id not in saio_historioak:
        saio_historioak[saio_id] = ChatMessageHistory()
    return saio_historioak[saio_id]

chain = txantiloia | llm
chain_memoriarekin = RunnableWithMessageHistory(
    chain, historia_lortu,
    input_messages_key="sarrera", history_messages_key="history"
)

config = {"configurable": {"session_id": "erabiltzaile-001"}}
print(chain_memoriarekin.invoke({"sarrera": "Kaixo! Nire izena Mikel da."},
config=config).content)
print(chain_memoriarekin.invoke({"sarrera": "Zer da nire izena?"}, config=config).content)
# "Zure izena Mikel da." → testuingurua gogoan du
```

`session_id` parametroak erabiltzaile bakoitzaren historia independentean kudeatzea ahalbidetzen du. Ekoizpenean, historia memorian gorde ordez, datu-base bati lotu beharko genioke (Redis edo PostgreSQL), memoria-jariotik babesteko.

🔍 ***Adibidea: laguntza-txatbota*** *Enpresa baten laguntza-txatbotak erabiltzaileak esandakoa gogoratu behar du elkarrizketan zehar. Erabiltzaileak "nire eskaera 1234 da" esaten badu eta gero "noiz iritsiko da?" galdetzen badu, txatbotak 1234 eskaerari buruz ari dela ulertu behar du.* `RunnableWithMessageHistory`*-rekin,* `session_id` *bakoitzeko historia mantentzen da, eta txatbotak testuingurua gogoan du.*

💡 ***Ideia nagusia*** *LLM-ak ez du oroimenik berez. Memoria mezuen historia gordez eta dei bakoitzean bidaliz lortzen da.* `session_id`*-k erabiltzaile bakoitzaren historia bereizten du.*

☐ ***Ariketa 5.3 · 25 min · banaka***

Eginkizuna*: Eraiki txatbot sinple bat, memoria duena, Gemini-rekin. Probatu hiru txandatan: aurkeztu zure izena, esan zer ikasten ari zaren, eta gero galdetu txatbotari "zer dakizu nitaz?". Egiaztatu gogoan duela.*

Eztabaidatzeko*: Zergatik bidali behar da historia osoa dei bakoitzean? Zer arazo sortzen du horrek elkarrizketa luzeetan? (Pista: token-kostua.)*

5.4 Tresnak eta agenteak

☐ ***Errepasoa: 5071 moduluan ikusi duzue eta 5. gaian sakontasunez ikusiko duzue*** *Agenteen kontzeptu orokorra eta enpresa-erabilera* ***5071 moduluan*** *landu dira (*`5071_2_AA_Enpresan` *— BBVA, Telefónica AURA, "esku-zubia-garuna" analogia,*

*automatizazioaren hiru mailak), eta **5. gai honetan** sakontasun handiarekin ikusiko dira (Google ADK, ReAct, plan-and-execute, multi-agent, ekoizpen-arazoak). Hemen LangChain-en **API teknikoan** zentratzen gara: tresnak Python funtzio gisa, `AgentExecutor`, `max_iterations`-en kontrola.*

LLM-en muga nagusietako bat da berek ez dutela kalkulatzeko, web-eko datuetara sartzeko edo API-ak deitzeko gaitasunik. Agentziek hori konpontzen dute: LLM-ari "tresna" sorta bat (Python funtzioak) eskaintzen zaio, eta berak erabakitzen du noiz eta zein deitu. Ahalmen handia ematen dute, baina arretaz erabili behar dira: `max_iterations`-ek begizta infinituak saihesten ditu.

```
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain.agents import create_tool_calling_agent, AgentExecutor
from langchain.tools import tool
from langchain.prompts import ChatPromptTemplate, MessagesPlaceholder
from datetime import datetime
import math

llm = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0)

@tool
def kalkulatu(eragiketa: str) -> str:
    """Matematikako eragiketa bat kalkulatu. Adibidez: '2**10', 'math.sqrt(16)'."""
    try:
        return f"Emaitza: {eval(eragiketa, {'math': math, '__builtins__': {}})}"
    except Exception as e:
        return f"Errorea: {e}"

@tool
def data_lortu(formatu: str = "%Y-%m-%d") -> str:
    """Gaur egungo data lortu. Formatua: '%Y-%m-%d' edo '%d/%m/%Y'."""
    return datetime.now().strftime(formatu)

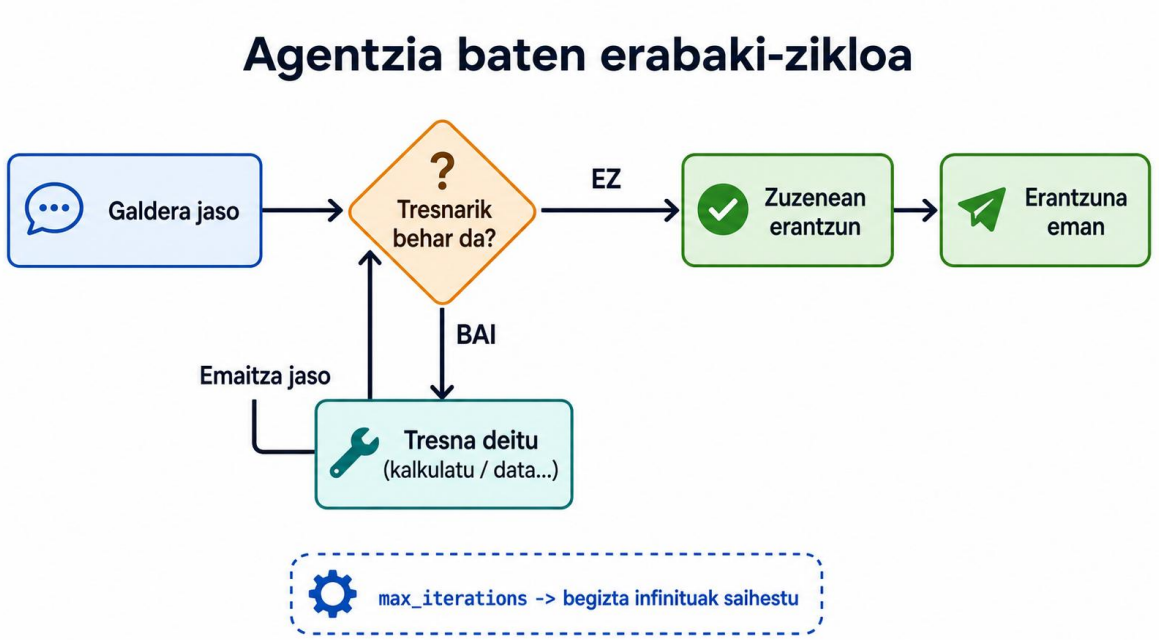
tresnak = [kalkulatu, data_lortu]

agente_txantiloia = ChatPromptTemplate.from_messages([
    ("system", "Laguntzaile adimendun eta zehatza zara. Beharrezkoak diren tresnak
erabili."),
    ("human",  "{sarrera}"),
    MessagesPlaceholder(variable_name="agent_scratchpad")
])

agentea = create_tool_calling_agent(llm, tresnak, agente_txantiloia)
exekutagailua = AgentExecutor(agent=agentea, tools=tresnak, verbose=True, max_iterations=5)

for galdera in ["Zenbat da 2 hamarrera berretuta?", "Gaur zein data da?"]:
    print(exekutagailua.invoke({"sarrera": galdera})["output"])
```

`@tool` dekoratzaileak Python funtzio bat tresna bilakatzen du, eta docstring-a deskribapen gisa erabiltzen du. Docstring argia funtsezkoa da: LLM-ak hori irakurriko du tresna noiz erabili erabakitzeko. `verbose=True`-k agentearen pentsamendu-prozesua erakusten du arazketarako.



20. Irudia: LLM agente baten erabaki-zikloa — galdera jaso, zein tresna erabili erabaki, tresna deitu, emaitza jaso, eta erantzuna eman, max_iterations mugarekin.

🔍 ***Adibidea: laguntzaile matematiko fidagarria*** *LLM-ek askotan huts egiten dute kalkulu konplexuetan (asmatu egiten dute). Agentea batekin, LLM-ak ez du kalkulatzen: `kalkulatu` tresna deitzen du, Python-ek kalkulatzen du zehazki, eta LLM-ak emaitza azaltzen du. Horrela, hizkuntzaren malgutasuna eta makinaren zehaztasuna konbinatzen dira.*

💡 ***Ideia nagusia*** *Agentziek LLM-ari tresnak (Python funtzioak) eskaintzen dizkiote, eta LLM-ak erabakitzen du noiz erabili. Docstring argia ezinbestekoa da: LLM-ak hori irakurtzen du tresna aukeratzeko.*

⚠ ***Kontuz: `eval()` segurtasun-arriskua*** *Adibideko `kalkulatu` funtzioak `eval()` erabiltzen du, eta hori arriskutsua da sarrera ez-fidagarriekin (kode arbitrarioa exekuta daiteke). Ekoizpenean, erabili parser seguru bat (`ast.literal_eval` edo liburutegi espezializatu bat), inoiz ez `eval()` hutsa erabiltzaile-sarrerarekin.*

☐ ***Ariketa 5.4 · 30 min · taldeka***

Eginkizuna*: Sortu agente bat bi tresnarekin: bat kalkulatzeko eta beste bat zerrenda bateko hitzak zenbatzeko. Probatu galdera nahasiekin ("Zenbat da 15*8 eta zenbat hitz ditu 'kaixo mundua'?").*

Eztabaidatzeko*: Nola erabakitzen du agenteak zein tresna deitu? Zer gertatuko litzateke docstring-a anbiguoa balitz?*

Atalaren laburpena: LangChain-ek GenAI aplikazioen oinarrizko piezak biltzen ditu: txantiloiak, chain-ak (LCEL), memoria eta agenteak, dena Gemini-rekin. Atal honetan ikusitakoarekin LLM-en gaineko aplikazio konplexuak modular eta mantengarriki eraiki daitezke. Hurrengo atalean (RAG) ezagutza-iturri kanpokoak gehituko dizkiogu LLM-ari.

6. RAG: Berreskurapenean-oinarritutako Sorkuntza

Atal honetan RAG (Retrieval-Augmented Generation) teknika ikasiko dugu, gaur egun LLM aplikazio gehienek erabiltzen duten patroia. Arrazoia argia da: RAG-ek LLM-ei aukera ematen die beren entrenamenduan ez zituzten dokumentu pribatuak, datu eguneratuak edo enpresa-jakintza ulertzeko, eta gainera "haluzinazioak" (asmakizunak) murrizten ditu, erantzunak iturri zehatzetan oinarrituz. Ikastaro honetan RAG-a **Gemini**rekin eraikiko dugu, embedding-etarako ere Google-ren ereduak erabiliz.

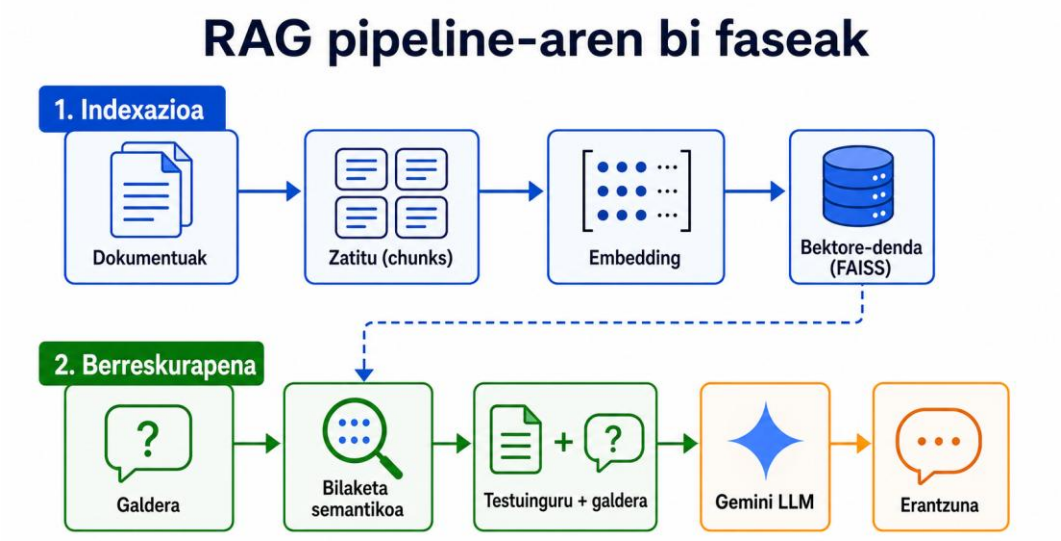
6.1 RAG zer da eta zergatik beharrezkoa den

☐ ***Errepasoa: 5071 moduluan ikusi duzue*** *RAG arkitekturaren kontzeptua eta zergatik beharrezkoa den* ***5071 moduluan*** *landu dira (`5071/01_NLP §01.15` — "RAG arkitektura: enpresaren ezagutza pribatua konektatu"). Hemen* ***inplementazio praktikoan*** *zentratzen gara: LangChain + Gemini + bektore-biltegiak (ChromaDB/FAISS), urratsez urrats kode funtzionalarekin 7. ataleko proiektu integratzaileraino.*

RAG-ek LLM-en muga nagusietako bat konpontzen du: LLM-ek entrenamendutik kanpoko informaziorik ez dakite (dokumentu pribatuak, datu eguneratuak). RAG-en ideia nagusia lau urratsekoa da:

1. **Dokumentuak bektore-biltegi batean gorde** (embedding bidez).
2. **Galdera bat jasotakoan**, dokumentu garrantzitsuenak bilatu semantikoki.
3. **Testuinguru gisa gehitu** LLM-ari bidalitako galderari.
4. **LLM-ak dokumentuetan oinarrituta erantzun**.

```
Erabiltzailea → Galdera
                    ↓
          Bektore-bilaketa (dokumentuak)
                    ↓
    [Galdera + Aurkitutako zatiak] → Gemini → Erantzuna
```



21. Irudia: RAG-en bi faseak — indexazioa (dokumentuak kargatu, zatitu, embed, gorde) eta berreskurapena (galdera, bilaketa semantikoa, testuinguru + galdera Gemini-ri).

TRESNA PRAKTIKOA: GOOGLE NOTEBOOKLM

RAG eskuz inplementatu aurretik, **[NotebookLM](https://notebooklm.google.com)** tresna erabilgarria da kontzeptua bisualki ulertzeko: dokumentuak igo, galderak egin, eta sistemak iturria aipatuz erantzuten du — koderik idatzi gabe. RAG + Gemini konbinatzen ditu, eta doakoa da. Ikasleentzat demo bikaina da RAG zer den ulertzeko.

RAG pipeline bat eraikitzeko paketeak (Gemini-rekin):

```
pip install langchain-google-genai langchain-community faiss-cpu pypdf
```

🔍 ***Adibidea: enpresaren eskuliburuari galdetzen*** *Enpresa batek 200 orriko eskuliburu bat dauka (prozedurak, politikak). Langile berriek galderak dituzte etengabe. RAG sistema batekin, eskuliburua bektore-biltegi batean gordetzen da, eta langileak galdetzen du "zenbat oporraldi-egun ditut?" — sistemak eskuliburuko atal zehatzetik erantzuten du, iturria aipatuz. ChatGPT orokorrak ezin du hori egin: ez du zure eskuliburua ezagutzen.*

💡 ***Ideia nagusia*** *RAG-ek LLM-ari kanpoko dokumentuak "irakurtzeko" aukera ematen dio: galdera jasotakoan dokumentu garrantzitsuenak bilatu eta testuinguru gisa eransten dizkio. Horrek haluzinazioak murrizten ditu eta iturriak aipatzeko aukera ematen du.*

☐ ***Ariketa 6.1 · 15 min · taldeka***

Eginkizuna*: Sartu [NotebookLM](https://notebooklm.google.com)-en, igo dokumentu bat (PDF edo testua) eta egin 3 galdera. Begiratu nola aipatzen dituen iturriak.*

> ***Eztabaidatzeko**: Zergatik ezin dio ChatGPT orokor batek zure dokumentu pribatuari buruz erantzun? Zer abantaila du RAG-ek?*

6.2 Dokumentuak kargatu eta zatitu

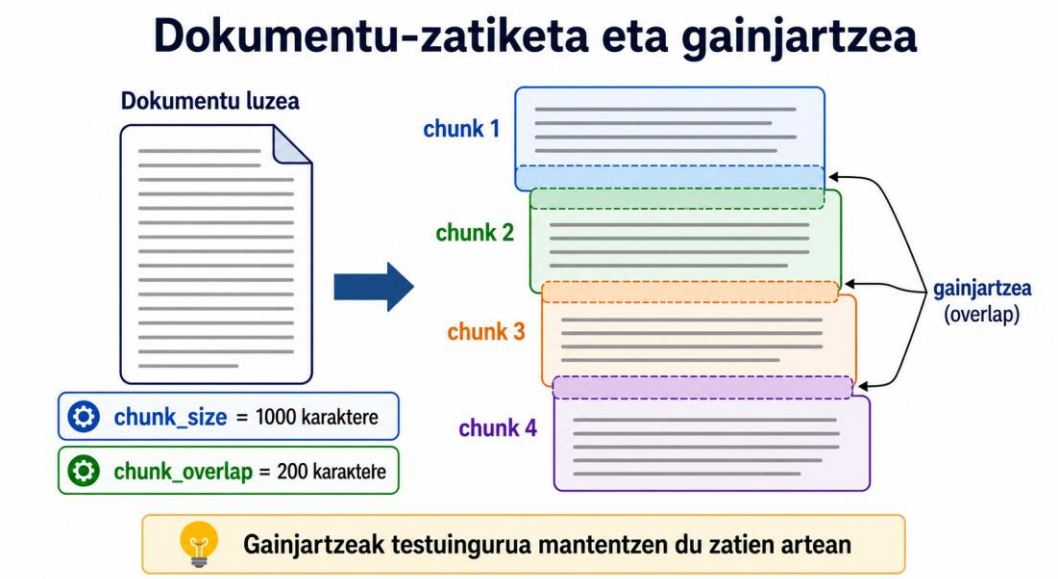
RAG pipeline orok bi atal ditu: **indexazioa** (dokumentuak prozesatu, zatitu, embed eta gorde) eta **berreskurapena** (galderaren araberako bilaketa eta erantzuna). Indexazio-fasean dokumentuak zuzenean ezin dira tratatu: LLM-ek testuinguru-leiho mugatua dute. Beraz, zati txikiagotan banatzen ditugu ("chunks"). `RecursiveCharacterTextSplitter` gehien gomendatutakoa da, paragrafoetan, esaldietan eta hitzetan modu hierarkikoan banatzen baitu.

```
from langchain_community.document_loaders import PyPDFLoader, TextLoader, WebBaseLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.schema import Document

# ---- Dokumentuak kargatu (hainbat iturri) ----
pdf_dok  = PyPDFLoader("txostena.pdf").load()
testu_dok = TextLoader("dokumentua.txt", encoding="utf-8").load()
web_dok  = WebBaseLoader("https://python.org/about/").load()

# ---- Dokumentuak zatitu (chunking) ----
zatitzailea = RecursiveCharacterTextSplitter(
    chunk_size=1000,        # Zati bakoitzeko gehienezko karaktere-kopurua
    chunk_overlap=200,      # Zatien arteko gainjartzea (testuingurua ez galtzeko)
    separators=["\n\n", "\n", ". ", " ", ""]
)
zatiak = zatitzailea.split_documents(pdf_dok)
print(f"Zatiak: {len(zatiak)}")
print(f"Lehen zatiaren metadatuak: {zatiak[0].metadata}")
```

`chunk_size` eta `chunk_overlap` doitzea kalitate-egokitzapen praktikoa da. Zati handiegiak (>1500) testuinguru ugari ematen dute baina leihoa azkar betetzen; txikiegiak (<300) testuinguru-galera eragin dezakete. Gainjartzeak zatien arteko jarraitutasuna mantentzen du. Metadatuak (iturria, orrialdea) gordetzea funtsezkoa da gero iturriak aipatzeko.



22. Irudia: Dokumentu luze bat zatitan (chunks) banatuta, chunk_overlap gainjartzearekin — nola mantentzen den testuingurua bi zatiren artean.

🔍 ***Adibidea: PDF luzea zatika*** *50 orriko kontratu bat duzu. LLM bati osorik bidaltzea ezinezkoa da (testuinguru-leihoa gainditzen du) eta garestia.* `RecursiveCharacterTextSplitter`*-ekin, kontratua 1.000 karaktereko zatitan banatzen duzu, 200eko gainjartzearekin. Galdera bat egitean, zati garrantzitsuenak soilik bidaltzen zaizkio Gemini-ri, ez kontratu osoa.*

💡 ***Ideia nagusia*** *Dokumentuak zati txikitan banatu behar dira (chunking) LLM-aren testuinguru-leihoan sartzeko. Gainjartzeak (overlap) zatien arteko jarraitutasuna mantentzen du. Metadatuak iturriak aipatzeko gorde.*

☐ ***Ariketa 6.2 · 20 min · banaka***

Eginkizuna*: Hartu testu luze bat (artikulu bat, 2-3 orrialde) eta zatitu* `RecursiveCharacterTextSplitter`*-ekin* `chunk_size=500`*-rekin. Aldatu* `chunk_overlap` *0, 100 eta 200 artean eta begiratu nola aldatzen den zati-kopurua.*

Eztabaidatzeko*: Zer arazo sor lezake gainjartzerik ez izateak? Eta gainjartze handiegiak?*

6.3 Bektore-biltegiak eta bilaketa semantikoa

Dokumentuak zatituta, esaldi bakoitza zenbaki-bektore bihurtu behar dugu, antzekotasun semantikoa neurtu ahal izateko. Embedding-eredu batek hori egiten du: testua bektore baten bidez ordezkatzen du, non esanahi antzeko testuek bektore antzekoak dituzten. Bektore-dendek (FAISS, Chroma, Pinecone) milioika bektore eraginkortasunez gordetzen eta bilatzen dituzte. Atal honetan FAISS

erabiliko dugu (tokian, memorian), prototipo eta tamaina ertaineko aplikazioetarako aproposa. Embedding-etarako Gemini-ren eredua erabiliko dugu.

```
from langchain_google_genai import GoogleGenerativeAIEmbeddings
from langchain_community.vectorstores import FAISS
from langchain.schema import Document

# ---- Embedding-ak: testua bektore zenbakira (esanahia mantenduz) ----
embedding_eredua = GoogleGenerativeAIEmbeddings(model="models/text-embedding-004")

dokumentuak = [
    Document(page_content="Scikit-Learn ikasketa automatikorako liburutegia da.",
metadata={"iturria": "1"}),
    Document(page_content="Random Forest hainbat erabaki-zuhaitzen multzoa da.",
metadata={"iturria": "2"}),
    Document(page_content="FastAPI REST API-ak azkar sortzeko esparrua da.",
metadata={"iturria": "3"}),
]

bektore_biltegia = FAISS.from_documents(dokumentuak, embedding_eredua)

# ---- Bilaketa semantikoa ----
emaitzak = bektore_biltegia.similarity_search("Zer da ikasketa automatikoa?", k=2)
for dok in emaitzak:
    print(f"(iturria {dok.metadata['iturria']}): {dok.page_content}")

# ---- Diskoan gorde eta retriever gisa erabili ----
bektore_biltegia.save_local("bektore_biltegia_faiss")
berreskuratzailea = bektore_biltegia.as_retriever(search_kwargs={"k": 3})
```

`k` parametroak zenbat dokumentu berreskuratzen diren kontrolatzen du. Balio txikiak (k=2-3) erantzun zehatzago eta merkeagoak; balio handiagoek (k=5-10) testuinguru aberatsagoa baina leihoa azkar betetzen. Garrantzitsua: bilaketa **semantikoa** da, ez gako-hitzekoa — "ikasketa automatikoa" galderak "machine learning" testua aurkituko luke, hitzak desberdinak izan arren.



23. Irudia: Bilaketa semantikoaren intuizioa — testuak bektore-espazio batean puntu gisa, eta esanahi antzeko testuak elkarrengandik gertu, galderatik hurbilen dauden zatiak hautatuz.

🔍 ***Adibidea: sinonimoak ulertzen*** *Erabiltzaileak "nola entrenatu eredu bat?" galdetzen du, baina dokumentuan "modeloen prestakuntza" jartzen du. Gako-hitzeko bilaketa batek ez luke aurkituko (hitzak desberdinak dira), baina bilaketa semantikoak bai: embedding-ek antzeko esanahia detektatzen dute. Hori da RAG-en magia: esanahiaren arabera bilatzen du, ez hitz zehatzen arabera.*

💡 ***Ideia nagusia*** *Embedding-ek testua bektore bihurtzen dute, esanahia mantenduz. Bilaketa semantikoak esanahiaren arabera aurkitzen ditu zatiak (ez gako-hitzen arabera), sinonimoak eta parafrasiak ulertuz. `k`-k berreskuratutako zati-kopurua kontrolatzen du.*

☐ ***Ariketa 6.3 · 20 min · taldeka***

***Eginkizuna**: Sortu 6-8 dokumentu (esaldi bana) gai desberdinekin. Eraiki FAISS biltegi bat Gemini embedding-ekin. Egin galdera bat sinonimoak erabiliz (dokumentuko hitz zehatzak ez) eta begiratu ea ondo aurkitzen duen.*

***Eztabaidatzeko**: Zergatik aurkitzen du bilaketa semantikoak sinonimoak gako-hitzeko bilaketak ez bezala?*

6.4 RAG pipeline osoa

Aurreko atalak elkartzen ditugu RAG chain oso batean: dokumentuen karga, zatiketa, embedding-en sorrera, bektore-biltegia eta Gemini-rekin kateaketa. LCEL erabiliz pipeline guztia deklaratiboki adierazten dugu, eta `RunnablePassthrough`-k sarrerako galdera txantiloira pasatzeko aukera ematen du. Prompt-ak garrantzia handia du: Gemini-ri espresuki esaten zaio testuinguruan informaziorik ez badago "Ez dakit" erantzuteko, haluzinazioak murrizteko.

```
from langchain_google_genai import ChatGoogleGenerativeAI, GoogleGenerativeAIEmbeddings
from langchain_community.vectorstores import FAISS
from langchain.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.schema import Document

# ---- 1. Dokumentuak prestatu ----
dokumentuak_gordinak = [
    Document(page_content="""FastAPI Python-en REST API-ak sortzeko esparru modernoa da.
    Pydantic-ekin datu-balioztatzea egiten du eta Swagger UI dokumentazioa sortzen du.""",
             metadata={"iturria": "fastapi_gida.txt"}),
    Document(page_content="""RAG metodoak LLM-ari kanpoko dokumentuetara sarbidea ematen
dio.
    Galdera bat jasotakoan, dokumentu garrantzitsuenak berreskuratzen dira.""",
             metadata={"iturria": "rag_gida.txt"}),
]

# ---- 2. Zatitu, embed eta bektore-biltegia ----
zatiak = RecursiveCharacterTextSplitter(chunk_size=500,
chunk_overlap=100).split_documents(dokumentuak_gordinak)
embedding_eredua = GoogleGenerativeAIEmbeddings(model="models/text-embedding-004")
bektore_biltegia = FAISS.from_documents(zatiak, embedding_eredua)
berreskuratzailea = bektore_biltegia.as_retriever(search_kwargs={"k": 3})

# ---- 3. Prompt template RAGerako (haluzinazioak murrizteko) ----
rag_txantiloia = ChatPromptTemplate.from_template("""
Beheko testuingurua erabiliz, galderari erantzun. Testuinguruan informazioa
ez badago, esan "Ez dakit, informazioa ez dago eskuragarri." Asmatu gabe erantzun.

Testuingurua:
{testuingurua}
```

```
Galdera: {galdera}

Erantzuna:""")

# ---- 4. RAG chain osoa (LCEL) ----
llm = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0)

def dok_formatu(dokumentuak):
    return "\n\n---\n\n".join(
        f"[Iturria: {d.metadata.get('iturria', 'ezezaguna')}]\n{d.page_content}"
        for d in dokumentuak)

rag_chain = (
    {"testuingurua": berreskuratzailea | dok_formatu, "galdera": RunnablePassthrough()}
    | rag_txantiloia | llm | StrOutputParser()
)

# ---- 5. Probatu ----
for galdera in ["Nola erabiltzen da FastAPI?", "Nola funtzionatzen du RAG-ek?",
                "Zein da Python-en asmatzailea?"]:   # azkena ez dago testuinguruan
    print(f"\n❓ {galdera}\n🤖 {rag_chain.invoke(galdera)}")
```

Azken galdera ("Zein da Python-en asmatzailea?") propio sartu da testuinguruan informaziorik gabe, RAG-ek haluzinazioak nola kontrolatzen dituen ikusteko: "Ez dakit" erantzun behar luke. Ekoizpenean, teknika aurreratuagoak daude: birmoldatze-bilaketa (reranking), bilaketa hibridoa (semantikoa + gako-hitzak), eta erantzunen ebaluazioa.

🔍 ***Adibidea: haluzinazioa saihestea*** *RAG sistema bat erabiltzaileari laguntzeko erabiltzen ari zara. Galdera bat egiten dute zure dokumentuetan inon ez dagoena. LLM "biluzi" batek zerbait asmatuko luke (haluzinazio arriskutsua). Prompt egokiarekin ("informaziorik ez badago, esan 'Ez dakit'"), Gemini-k zintzoki aitortzen du ez dakiela. Hori funtsezkoa da konfiantzazko sistemetan: hobe "ez dakit" esatea gezurra esatea baino.*

💡 ***Ideia nagusia*** *RAG chain osoa LCEL-ekin deklaratiboki kateatzen da: berreskuratzailea + prompt + Gemini + parser. Prompt-ean "informaziorik ez badago, esan ez dakizula" jartzea funtsezkoa da haluzinazioak murrizteko.*

⚠ ***Kontuz: RAG-ek ez du dena konpontzen*** *RAG-ek ez du LLM-aren akats guztiak konpontzen. Berreskuratutako zatiak okerrak badira (chunking txarra, embedding eskasa), erantzuna ere okerra izango da. "Garbage in, garbage out": dokumentuen kalitatea eta zatiketa egokia funtsezkoak dira.*

☐ ***Ariketa 6.4 · 30 min · taldeka***

***Eginkizuna**: Eraiki RAG chain oso bat Gemini-rekin: 4-5 dokumentu, zatitu, embed, FAISS biltegia eta chain-a. Probatu galdera batekin testuinguruan dagoena eta beste batekin ez dagoena. Egiaztatu bigarrenean "Ez dakit" erantzuten duela.*

***Eztabaidatzeko**: Zer gertatuko litzateke prompt-ean "ez dakit" agindua kenduko bagenu? Zergatik da garrantzitsua iturriak aipatzea?*

Atalaren laburpena: RAG patroia LLM aplikazio modernoen oinarrizko teknika da. Dokumentuen kargatik galdera-erantzun sistema osoraino joan gara, FAISS bektore-biltegia eta LangChain LCEL erabiliz, dena Gemini-rekin. Patroi hau ezagututa, edozein dokumentu-iturri LLM batera ekartzeko gai izan beharko zinatekete, enpresetan oso eskatua.

---

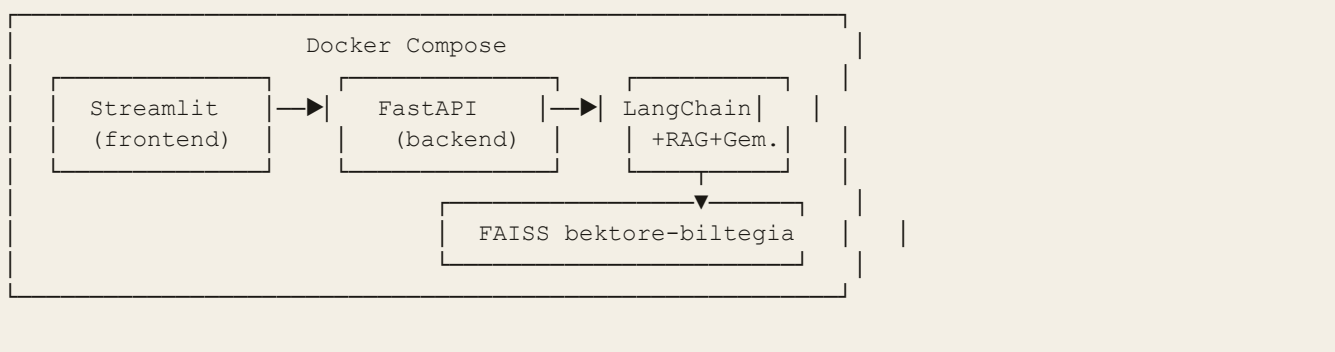
7. Proiektu Integratzailea: GenAI Aplikazio Osoa

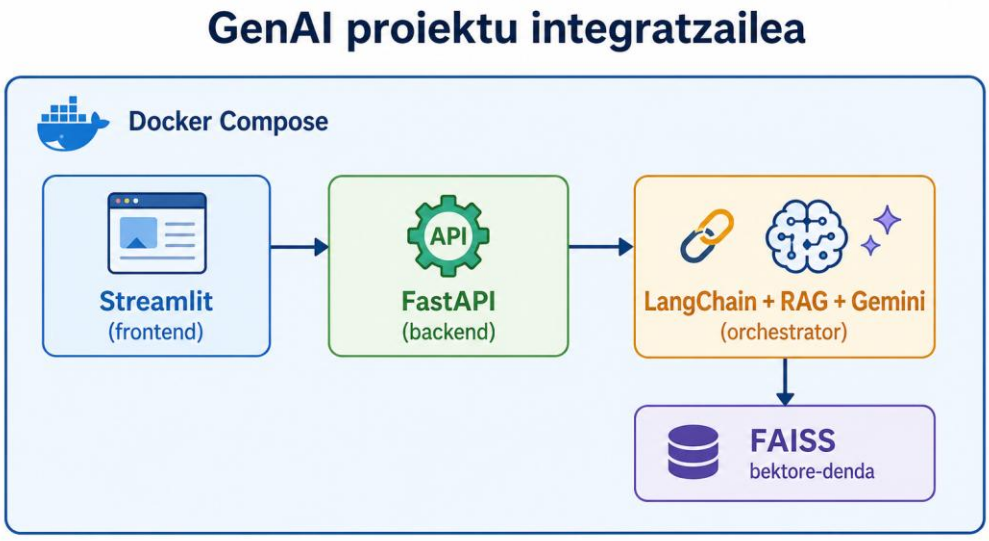
Atal honetan gaiko atal guztiak konbinatuko ditugu proiektu integratu batean: dokumentuen gaineko galdera-erantzun sistema bat, **FastAPI** (backend), **LangChain + RAG + Gemini** (logika), **Streamlit** (interfazea) eta **Docker** (hedapena) erabiliz. Arkitektura hau gaur egungo enpresa-inguruneetan eskatzen den patroi tipikoa da, eta atal guztietan ikasitakoa praktikan jartzeko aukera ematen du.

Helburu zehatza: **dokumentuen gaineko galdera-erantzun sistema** eraikitzea, non erabiltzaileak fitxategiak igo eta horien gainean galderak egin ditzakeen, sistemak iturriak aipatuz erantzunez.

Arkitektura

Arkitekturak hiru osagai nagusi ditu: frontend-a (Streamlit), backend-a (FastAPI) eta RAG motorra (LangChain + FAISS + Gemini). Hirurak Docker Compose bidez orkestratzen dira, eta osagai bakoitzak bere ardura zehatza du, sistema modular eta eskalagarria sortuz.





24. Irudia: GenAI proiektu integratzailearen arkitektura osoa — Streamlit frontend-a, FastAPI backend-a, LangChain + RAG + Gemini motorra eta FAISS biltegia, Docker Compose bidez orkestratuta.

rag_api.py — Backend osoa (Gemini-rekin)

Backend-ak bi ibilbide eskaintzen ditu: dokumentuak kargatzeko (`/dokumentuak/kargatu`) eta galderak egiteko (`/galdera`). Egoera memorian gordetzen du sinpletasunagatik; ekoizpen errealean bektore-biltegi iraunkor bat erabili beharko litzateke (Chroma diskoan, edo Pinecone/Weaviate).

```
from fastapi import FastAPI, HTTPException, UploadFile, File
from pydantic import BaseModel
from langchain_google_genai import ChatGoogleGenerativeAI, GoogleGenerativeAIEmbeddings
from langchain_community.vectorstores import FAISS
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough
from langchain.schema import Document
from typing import Optional

app = FastAPI(title="Dokumentuen Galdera-Erantzun Sistema")

bektore_biltegia: Optional[FAISS] = None
embedding_eredua = GoogleGenerativeAIEmbeddings(model="models/text-embedding-004")
llm = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0)

class GalderaEskaera(BaseModel):
    galdera: str
    k_dokumentu: int = 3

class GalderaErantzuna(BaseModel):
    erantzuna: str
    iturriak: list[str]
    dokumentu_kopurua: int

@app.post("/dokumentuak/kargatu")
async def dokumentuak_kargatu(fitxategiak: list[UploadFile] = File(...)):
    global bektore_biltegia
    dokumentuak = []
    for f in fitxategiak:
        edukia = (await f.read()).decode("utf-8", errors="ignore")
        dokumentuak.append(Document(page_content=edukia, metadata={"iturria": f.filename}))
    zatiak = RecursiveCharacterTextSplitter(chunk_size=1000,
chunk_overlap=200).split_documents(dokumentuak)
    if bektore_biltegia is None:
        bektore_biltegia = FAISS.from_documents(zatiak, embedding_eredua)
    else:
        bektore_biltegia.add_documents(zatiak)
    return {"mezua": f"{len(fitxategiak)} fitxategi kargatuta", "zati_kopurua": len(zatiak)}

@app.post("/galdera", response_model=GalderaErantzuna)
def galdera_egin(eskaera: GalderaEskaera):
    if bektore_biltegia is None:
        raise HTTPException(status_code=400, detail="Dokumenturik ez dago kargatuta.")
    berreskuratzailea = bektore_biltegia.as_retriever(search_kwargs={"k":
```

```
eskaera.k_dokumentu})
    rag_txantiloia = ChatPromptTemplate.from_template("""
Beheko testuingurua erabiliz, galderari zehaztasunez erantzun.
Testuinguruan informazioa ez badago, esan hori argi.

Testuingurua:
{testuingurua}

Galdera: {galdera}

Erantzuna:""")
    berreskuratutako_dok = []
    def gorde_eta_formatu(dok_zerrenda):
        berreskuratutako_dok.extend(dok_zerrenda)
        return "\n\n".join(d.page_content for d in dok_zerrenda)
    chain = (
        {"testuingurua": berreskuratzailea | gorde_eta_formatu, "galdera":
RunnablePassthrough()}
        | rag_txantiloia | llm | StrOutputParser()
    )
    erantzuna = chain.invoke(eskaera.galdera)
    iturriak = list({d.metadata.get("iturria", "ezezaguna") for d in berreskuratutako_dok})
    return GalderaErantzuna(erantzuna=erantzuna, iturriak=iturriak,
                            dokumentu_kopurua=len(berreskuratutako_dok))

@app.get("/osasuna")
def osasuna():
    return {"egoera": "martxan", "dokumentuak_kargatuta": bektore_biltegia is not None}
```

Alderdi azpimarragarriak: `UploadFile`-ekin fitxategi anitz jasotzen ditu; `add_documents`-ekin dokumentu berriak existitzen den biltegiari gehitzen zaizkio; berreskuratutako dokumentuak gordetzen dira erantzunarekin batera iturriak itzultzeko (RAG sistemetan funtsezko ezaugarria).

rag_streamlit.py — Frontend-a

Frontend-a Streamlit-ekin eraikitzen da eta `requests` liburutegia erabiltzen du backend-aren API-ari deitzeko. Egitura sinplea da: sidebar-ean fitxategiak igotzeko atala, eta orri nagusian galderak egiteko interfazea. `API_URL` ingurune-aldagai gisa konfigura daiteke, Docker Compose-rekin integratzea errazteko.

```
import os
import streamlit as st
import requests

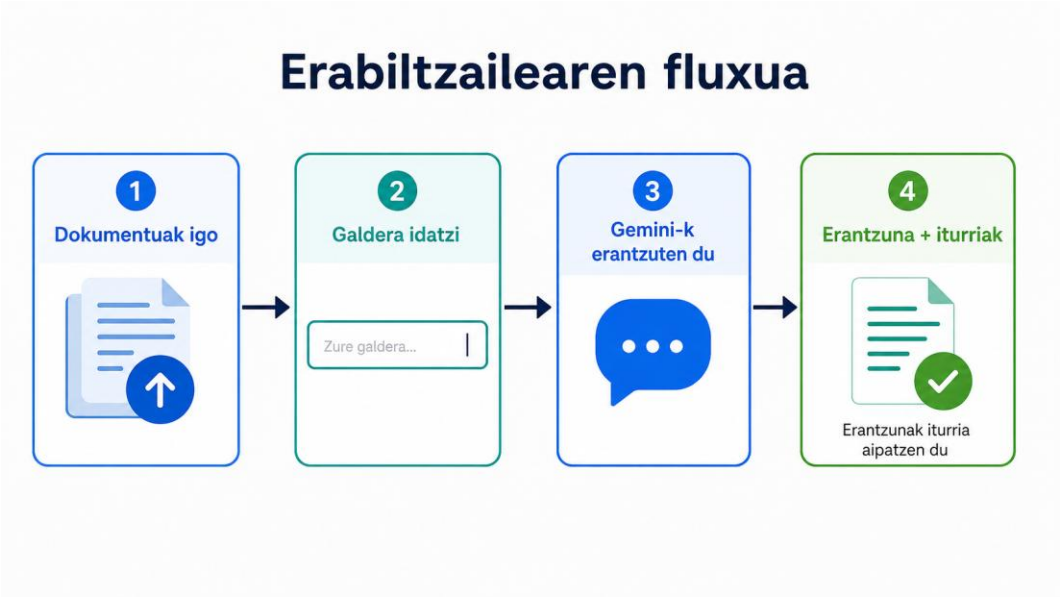
st.set_page_config(page_title="Dokumentuen Galdera-Erantzun Sistema", page_icon="□",
layout="wide")
API_URL = os.getenv("API_URL", "http://localhost:8000")

st.title("□ Dokumentuen Galdera-Erantzun Sistema")
st.markdown("Zure dokumentuak igo eta galderak egin (Gemini-z elikatua).")
```

```
with st.sidebar:
    st.header("□ Dokumentuak")
    igotakoak = st.file_uploader("Fitxategiak (TXT)", type=["txt"],
accept_multiple_files=True)
    if igotakoak and st.button("Kargatu", type="primary"):
        fitx = [("fitxategiak", (f.name, f.read(), "text/plain")) for f in igotakoak]
        try:
            r = requests.post(f"{API_URL}/dokumentuak/kargatu", files=fitx, timeout=60)
            if r.status_code == 200:
                d = r.json()
                st.success(f"✓ {d['mezua']} ({d['zati_kopurua']} zati)")
            else:
                st.error(f"Errorea: {r.text}")
        except requests.exceptions.ConnectionError:
            st.error("APIarekin konektatu ezin izan da.")

galdera = st.text_area("Galdera idatzi:", placeholder="Adibidez: Zer da RAG?", height=100)
k_dok = st.number_input("Dokumentu kopurua", min_value=1, max_value=10, value=3)

if st.button("Galdetu", type="primary") and galdera:
    with st.spinner("Erantzuna bilatzen..."):
        try:
            r = requests.post(f"{API_URL}/galdera",
                              json={"galdera": galdera, "k_dokumentu": k_dok}, timeout=60)
            if r.status_code == 200:
                d = r.json()
                st.subheader("□ Erantzuna"); st.write(d["erantzuna"])
                st.info(f"□ Iturria(k): {', '.join(d['iturriak'])}")
            elif r.status_code == 400:
                st.warning("□ " + r.json()["detail"])
        except requests.exceptions.ConnectionError:
            st.error("APIarekin konektatu ezin izan da.")
```



25. Irudia: Erabiltzailearen fluxua proiektu integratzailean — dokumentuak igo, galdera idatzi, sistemak Gemini-rekin erantzuten du eta iturriak erakusten ditu.

Frontend-aren funtzionamendua: erabiltzaileak fitxategiak igotzen ditu eta "Kargatu" sakatzen du (`multipart/form-data` eskaera). Galdera bat egiteko, testua idatzi eta `k` parametroa hautatzen du. Erantzun-egoeraren araberako mezuak erakusten dira: arrakasta (200), arazo logikoa (400, dokumenturik ez), edo konexio-errorea.

🔍 ***Adibidea: enpresarako prototipo osoa*** *Sarrerako eszenako enpresak orain tresna oso bat dauka: langileek dokumentuak igotzen dituzte (politikak, eskuliburuak), eta edozeinek galdetu diezaieke nabigatzailetik. Backend-ak Gemini-rekin erantzuten du, iturriak aipatuz. Dena Docker-en bilduta, edozein zerbitzaritan hedatzeko prest. Hori da gai osoaren emaitza: ariketa solteetatik produktu funtzional batera.*

💡 ***Ideia nagusia*** *Arkitektura modularra (frontend + backend + RAG + Docker) ekoizpen-aplikazioen oinarrizko patroia da. Osagai bakoitzak ardura bat du, eta horrek sistema mantengarri eta eskalagarria egiten du.*

☐ ***Hausnartzeko*** *Proiektu honi zer hobekuntza gehituko zenizkioke ekoizpenerako? (Pista: erabiltzaile-autentifikazioa, datu-base iraunkorra, monitorizazioa, CI/CD pipeline-a.) Zein litzateke lehentasuna enpresa txiki batean?*

☐ ***Ariketa 7.1 · 60 min · taldeka***

Egoera*: Talde bakoitzak gai bat aukeratzen du (errezetak, lege-testuak, ikasgaiaren apunteak…).*

Eginkizuna*: Eraiki proiektu integratzaile sinplifikatu bat: FastAPI backend bat RAG + Gemini-rekin eta Streamlit frontend bat. Kargatu 3-4 dokumentu eta egin galderak. Egiaztatu iturriak aipatzen dituela.*

Eztabaidatzeko*: Zein izan da zatirik zailena? Nola integratu dituzue atal guztiak? Zer aldatuko zenukete ekoizpenerako?*

Atalaren laburpena: Proiektu integratzaile honek gaian zehar ikasitako kontzeptu guztiak elkartzen ditu produktu funtzional batean. Arkitektura modularra ekoizpen-aplikazioen oinarrizko patroia da, eta hori barneratzeak enpresa-mailako GenAI soluzioak diseinatzeko gai egiten zaitu.

Laburpena eta gakoak

Gai honek ML eredu klasikoetatik GenAI aplikazio osoetara eraman gaitu, ibilbide praktiko batean. **Scikit-Learn**ekin ikasketa automatiko klasikoa landu dugu (aurreprozesatzea, sailkapena, erregresioa, pipeline-ak, ebaluazioa eta hiperparametro-doiketa), beti tresnen erabilera praktikoa lehenetsiz formula matematikoen gainetik. **Hugging Face**rekin eredu aurreentrenatuen mundua zabaldu dugu, `pipeline` API-arekin minutu gutxitan prototipoak eraikiz. **FastAPI**rekin ML ereduak web-zerbitzu profesional bihurtzen ikasi dugu, eta **Streamlit**ekin emaitzak interaktiboki aurkezten. **LangChain + Gemini**rekin LLM aplikazioak (chain-ak, memoria, agenteak) eraiki ditugu, eta **RAG** patroiarekin dokumentu propioei galderak egiteko sistemak. Azkenik, dena **proiektu integratzaile** batean batu dugu.

Hau da hurrengo gaiaren oinarria: 4. gaian integrazioa, hedapena eta GenAI sakonago landuko ditugu, agente korporatiboetara igaroz. Gai honetan ikasitako tresna bakoitza —Scikit-Learn, FastAPI, Streamlit, LangChain, RAG— hurrengo proiektuetan etengabe erabiliko duzu: hauek dira IA-teknikari baten eguneroko lan-tresnak.

> 💡 ***Ideia nagusi*** *IA-teknikari profesionala ez da algoritmo bat entrenatzen dakiena soilik — eredua produktu funtzional bihurtzen dakiena ere bada: API bat, web-aplikazio bat, dokumentuei galdetzeko sistema bat. Gai honek "eredua dauka" mailatik "edozeinek erabil dezakeen produktua dauka" mailara eramaten zaitu.*
>
> ☐ ***Hausnartzeko*** *Hurrengo proiektu pertsonal batean, zein tresna konbinatuko zenituzke? Adibidez: scikit-learn eredu bat entrenatu, FastAPI-rekin zerbitzaratu, Streamlit-ekin interfazea eman, eta agian RAG bat gehitu dokumentazioarentzat. Zer ordenatan eraikiko zenuke? Zer probatuko zenuke lehenik?*

Erreferentziak

Dokumentazio ofiziala

- [Scikit-Learn dokumentazioa](https://scikit-learn.org/stable/user_guide.html)
- [Hugging Face Transformers dokumentazioa](https://huggingface.co/docs/transformers)
- [Hugging Face Hub](https://huggingface.co/models)
- [FastAPI dokumentazioa](https://fastapi.tiangolo.com/)
- [Streamlit dokumentazioa](https://docs.streamlit.io/)
- [LangChain dokumentazioa](https://python.langchain.com/docs/introduction/)
- [LangChain Google GenAI integrazioa](https://python.langchain.com/docs/integrations/chat/google_generative_ai/)
- [Google AI Studio (Gemini API-gakoa)](https://aistudio.google.com/apikey)
- [Google Gemini API dokumentazioa](https://ai.google.dev/gemini-api/docs)

Tutorialak eta baliabide osagarriak

- [Scikit-Learn: Adibideak eta tutorialak](https://scikit-learn.org/stable/auto_examples/)
- [Hugging Face Ikastaroa (doakoa)](https://huggingface.co/learn/nlp-course)
- [FastAPI tutoriala (urratsez urrats)](https://fastapi.tiangolo.com/tutorial/)
- [Streamlit Gallery (adibideak)](https://streamlit.io/gallery)
- [Google NotebookLM](https://notebooklm.google.com) — RAG demo bisuala
- [Hands-On ML with Scikit-Learn, Keras & TF (Géron)](https://www.oreilly.com/library/view/hands-on-machine-learning/9781098125967/)

Eredu eta tresna euskaraz

- [Hugging Face: Euskarazko ereduak](https://huggingface.co/models?language=eu)
- [BERT eredu eleaniztuna](https://huggingface.co/bert-base-multilingual-cased)
- [IXA taldea (EHU) — Euskarazko NLP](https://ixa.eus/)

Lege-erreferentziak

- Errege Dekretu 279/2021 — Adimen Artifizial eta Big Data Espezializazio-ikastaroaren oinarrizko curriculuma (IE2).
